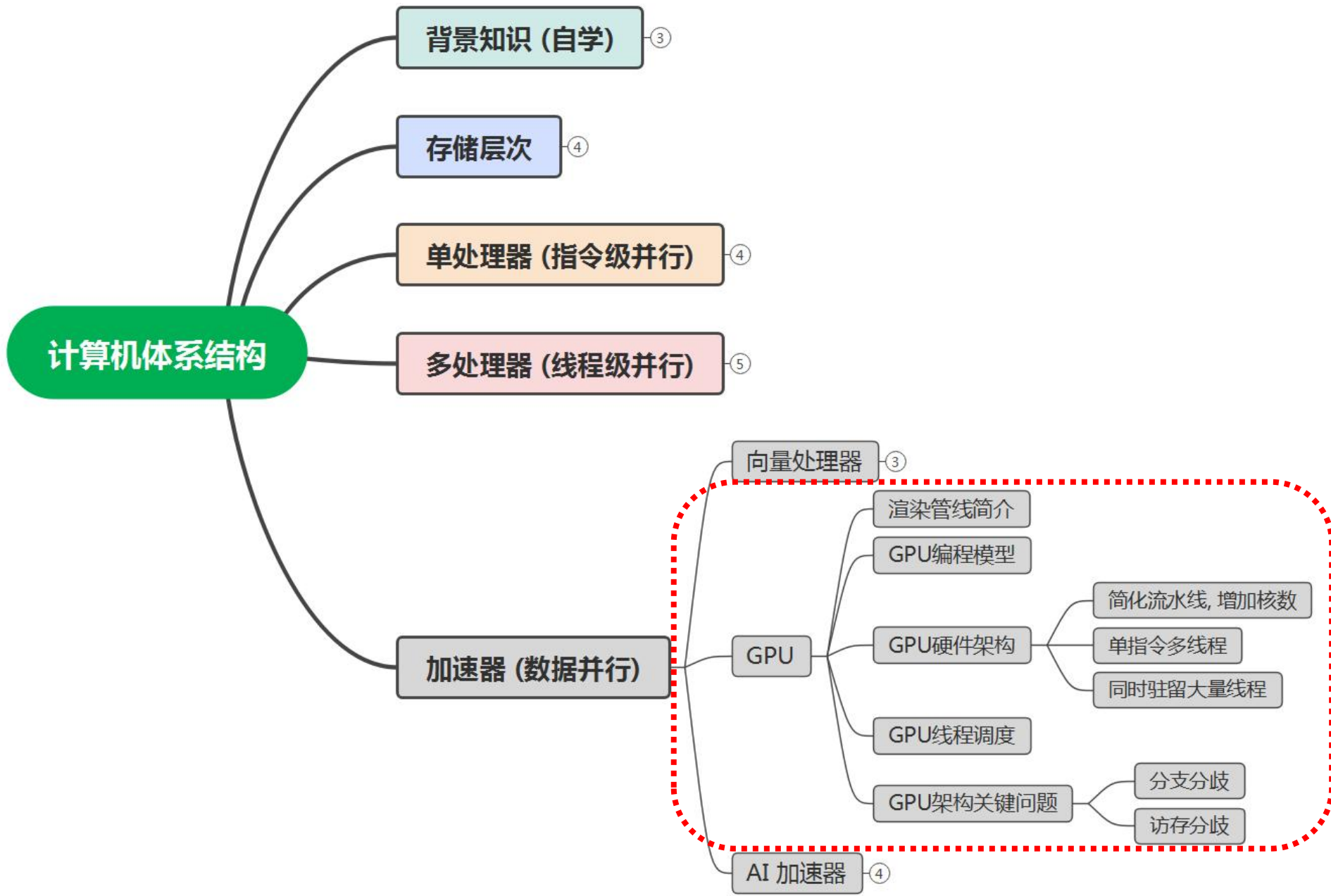


计算机体系结构

L08 GPU



为何学习GPU?

□ 非常成功的通用加速器/协处理器

□ GPU结合了两种策略来提高效率

- 大规模并行
- 定制化

□ 展示了加速器中性能和可编程性之间的矛盾关系

现代计算机中的GPU

□ 独立GPU

- 基于PCIe的加速器
- 独立显存

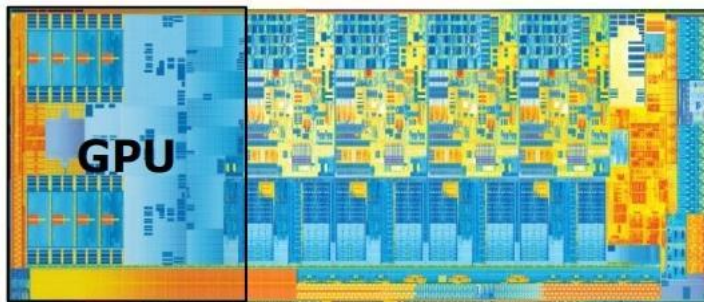


Nvidia Kepler

□ 集成GPU

- CPU和GPU在一个芯片上
- 共享主内存和LLC

□ 优缺点?



Intel Ivy Bridge, 22nm 160mm²



Apple A7, 28nm

GPU 演变过程

□ 第一阶段：图形专用固定功能管线（1990s）

- 专为3D图形渲染设计
- 固定功能硬件单元（顶点变换、光栅化、纹理映射）
- 管线不可编程，仅通过API配置

□ 第二阶段：可编程着色器崛起（2000-2006）

- 引入可编程着色器单元
- 支持DirectX 8.0/9.0和OpenGL着色语言
- 从固定功能向有限可编程转变

□ 第三阶段：统一架构与通用计算（2006-2012）

- 统一着色器架构：顶点、像素、几何着色器使用相同处理单元
- CUDA诞生：首次将GPU作为通用并行处理器
- GPGPU普及：OpenCL（2009）、DirectCompute发布

GPU 演变过程

□ 第四阶段：并行计算与架构优化（2012-2017）

- 能效大幅提升：每瓦性能成为重点
- 内存层次优化：L1/L2缓存、共享内存改进
- 双精度计算支持：面向HPC市场

□ 第五阶段：AI与张量计算时代（2017-2020）

- 专用AI硬件：张量核心加速矩阵运算
- 实时光线追踪：专用RT核心
- 深度学习爆发：GPU成为AI训练基础设施

□ 第六阶段：领域专用与超大规模（2020至今）

- 芯片化（Chiplet）：多芯片模块设计
- 内存创新：HBM2e/HBM3、显存带宽突破

GPU用于图像渲染

图像渲染

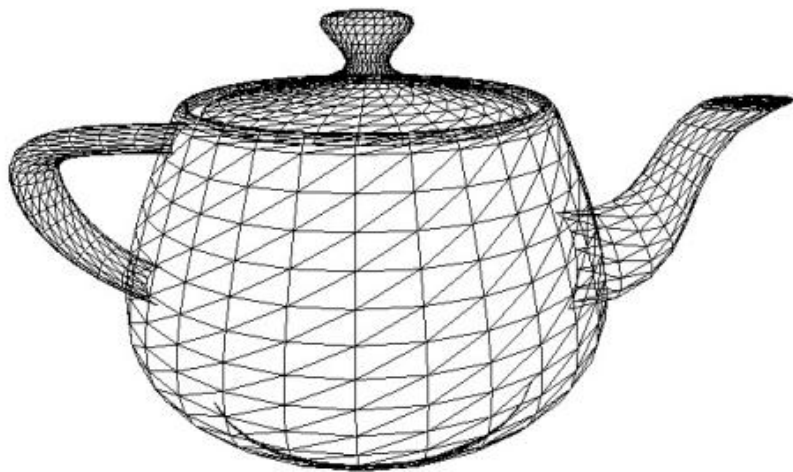


Image credit: Henrik Wann Jensen

输入: 场景描述

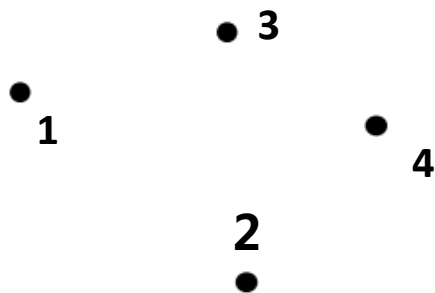
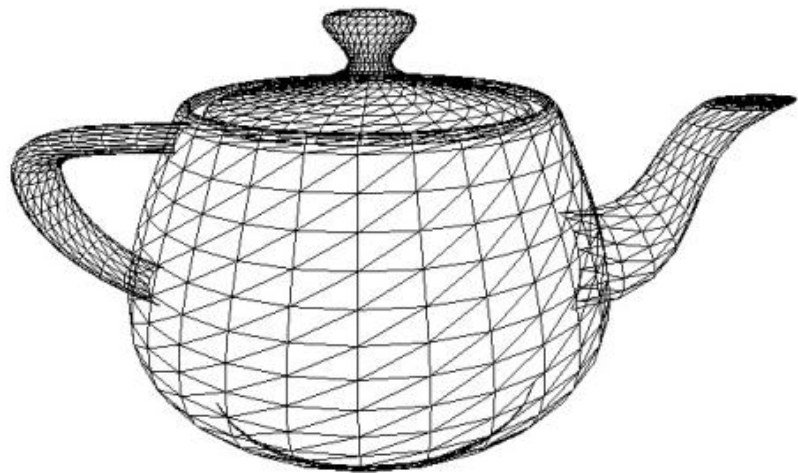
3D表面几何（例如，三角网格）、
表面材质、光源、相机等

输出: 场景图片

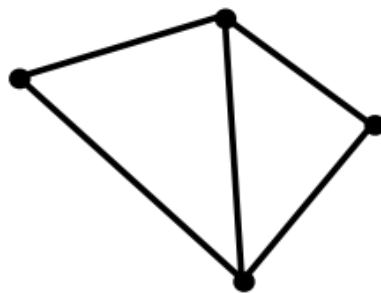
渲染任务的简单定义：将由三角形等特征表示的3D场景，转化为
由像素组成的图片

基本术语

将表面表示为三角形网格(3D mesh)

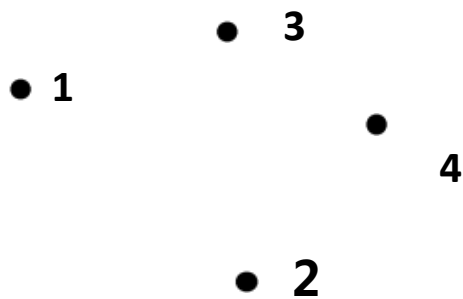


顶点(Vertexes)

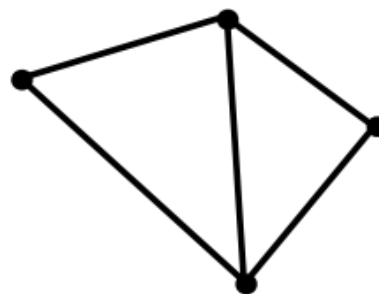


图元(Primitives)
(e.g., 三角形, 点, 线)

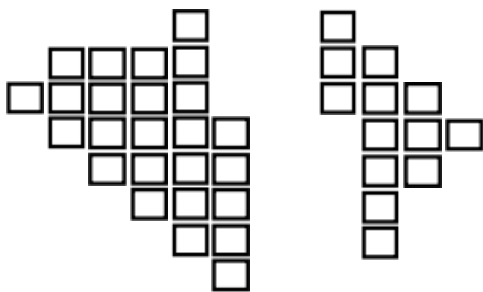
基本术语



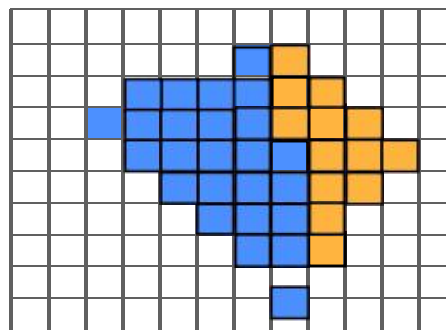
顶点(Vertexes)



图元(Primitives)
(e.g., 三角形, 点, 线)



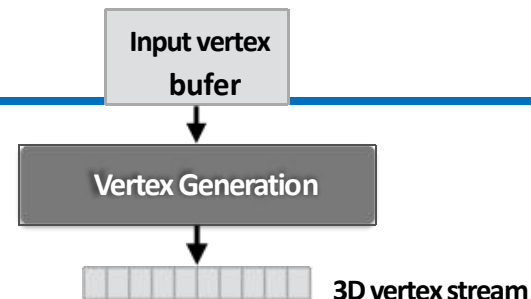
片段 (fragment)



像素(Pixels)

渲染流程

输入: 3D空间中的一些顶点(以及他们之间如何组成图元的信息)



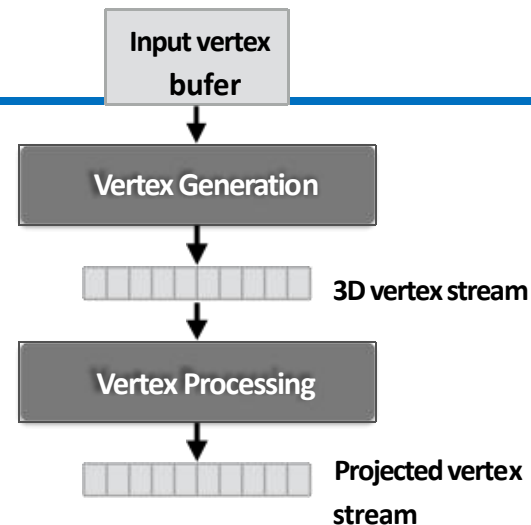
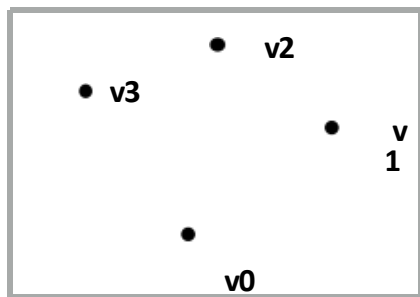
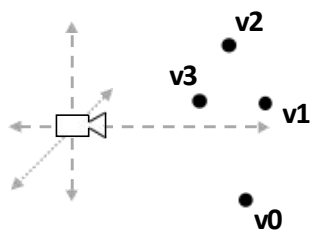
例如: 每3个顶点组成一个三角形

```
list_of_positions =  
{  
    v1x, v1y, v1x, }  
    v0x, v0y, v0z, } triangle 0 = {v0, v1, v2}  
    v2x, v2y, v2z, }  
    v3x, v3y, v3x } triangle 1 = {v1, v2, v3}  
};
```

The code snippet shows a list of vertex positions. Blue arrows indicate how these positions are grouped into triangles. The first three positions (v1x, v1y, v1x) are grouped with v0x, v0y, v0z to form 'triangle 0 = {v0, v1, v2}'. The next three positions (v2x, v2y, v2z) are grouped with v1x, v1y, v1x to form 'triangle 1 = {v1, v2, v3}'.

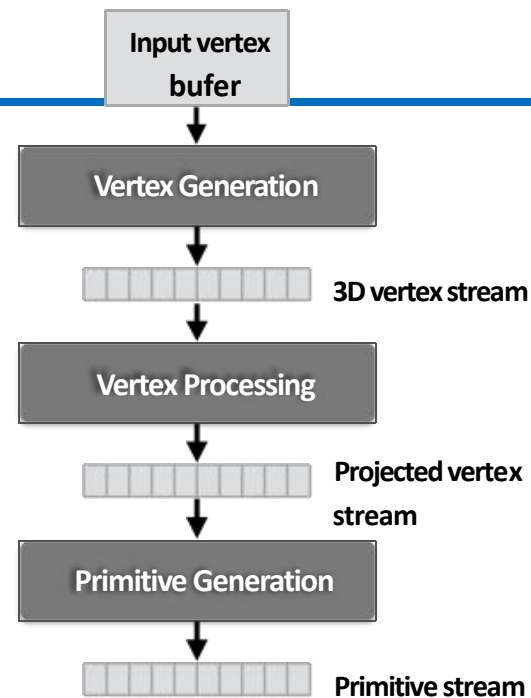
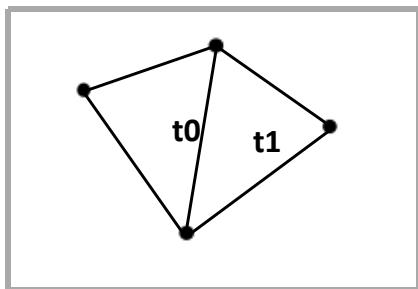
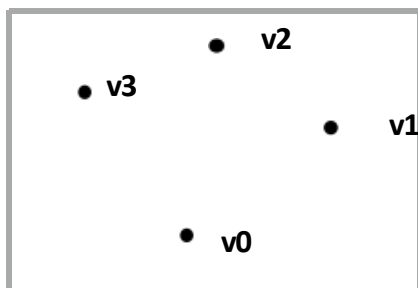
渲染流程

Step 1: 给定场景中的相机位置，计算顶点在屏幕上的位置



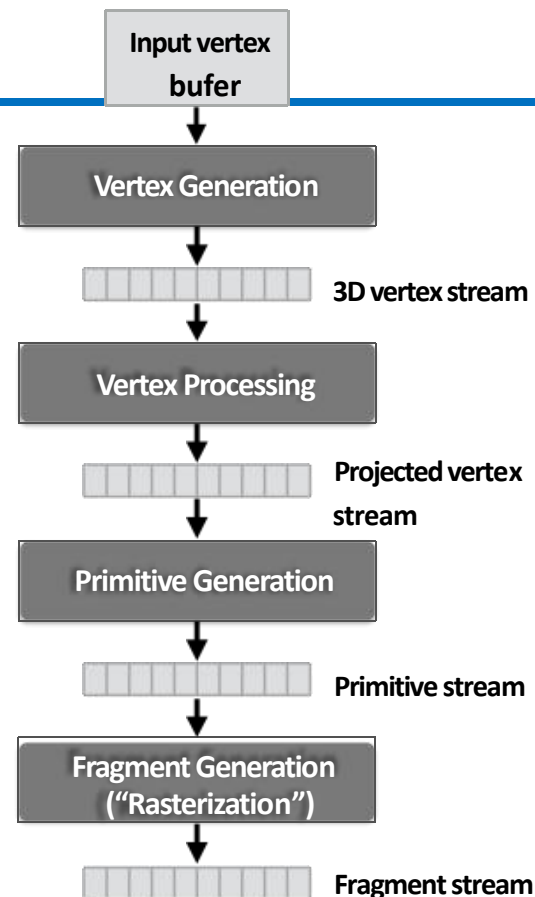
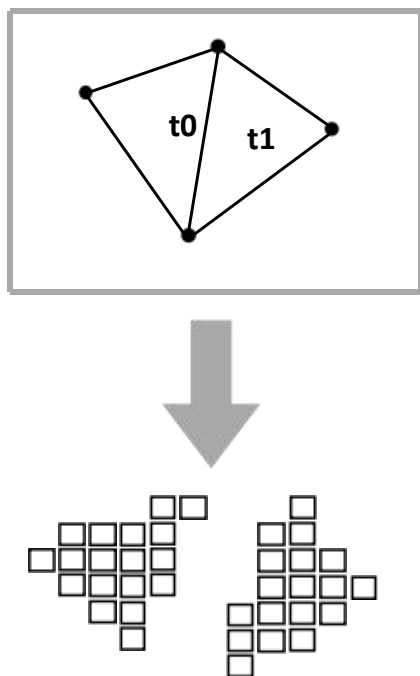
渲染流程

Step 2: 将顶点组织成图元(e.g.,三角形)



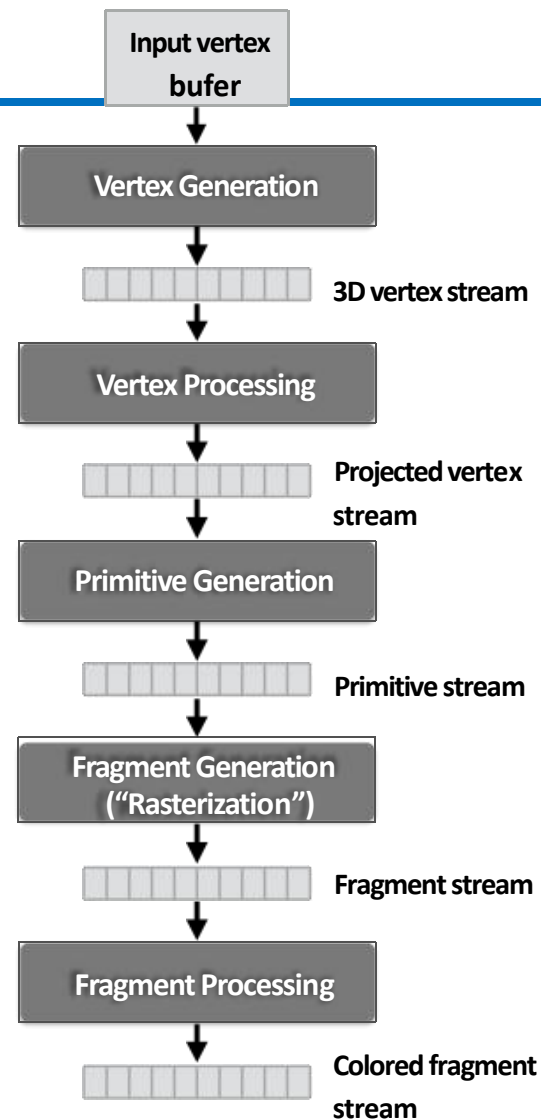
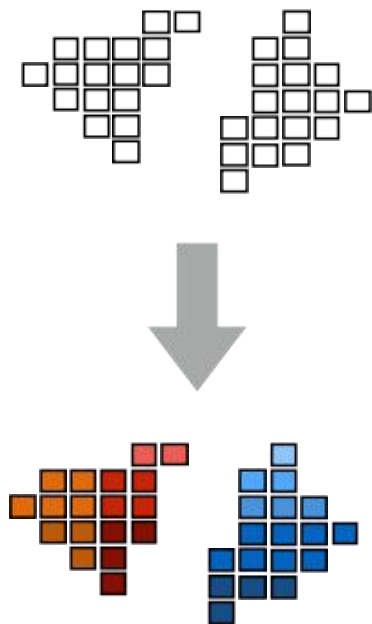
渲染流程

Step 3: 根据图元生成片段，每个片段对应一个像素位置



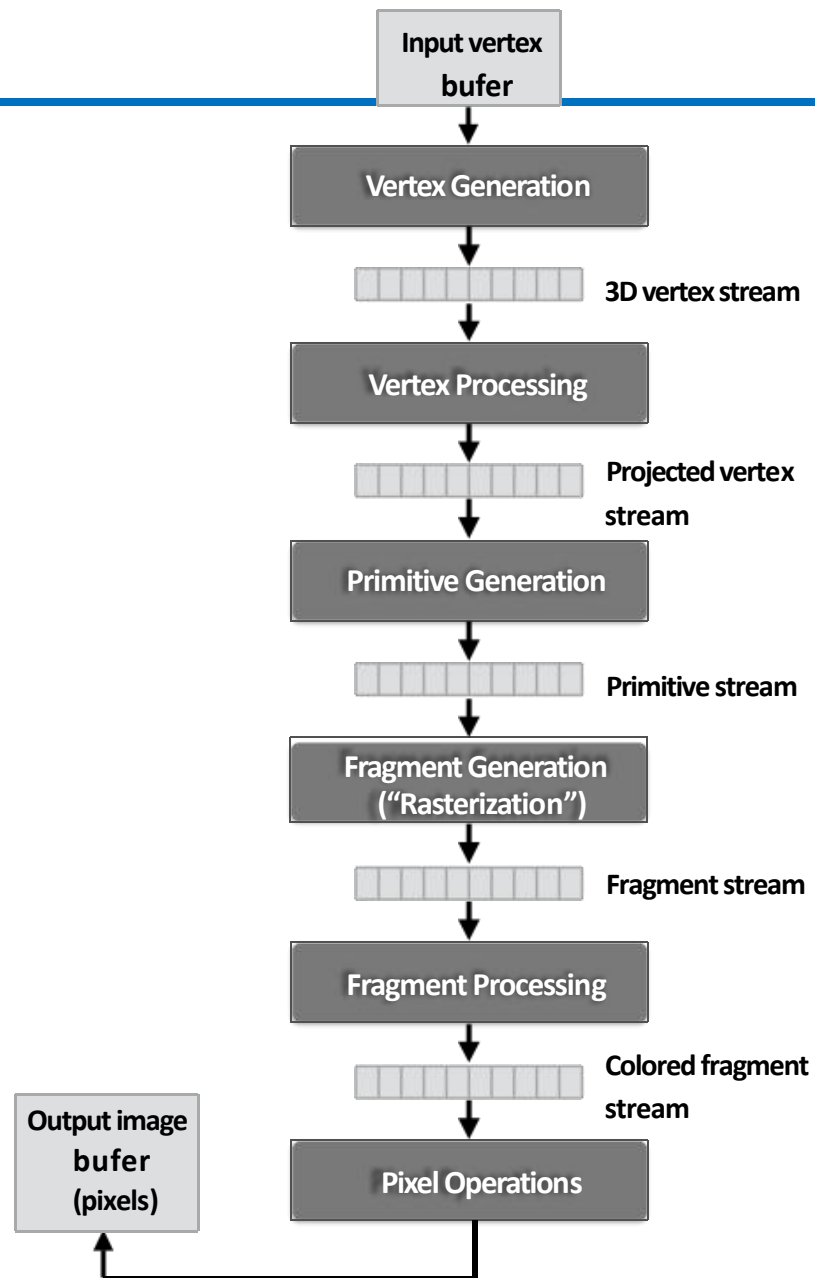
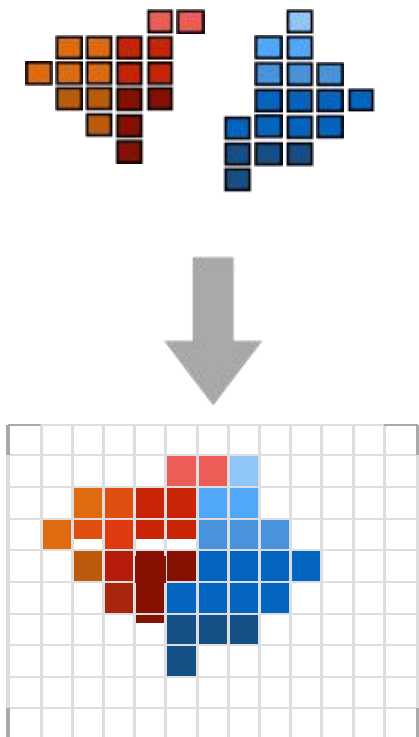
渲染流程

Step 4: 根据场景光照和图元材质属性，计算每个片段的颜色



渲染流程

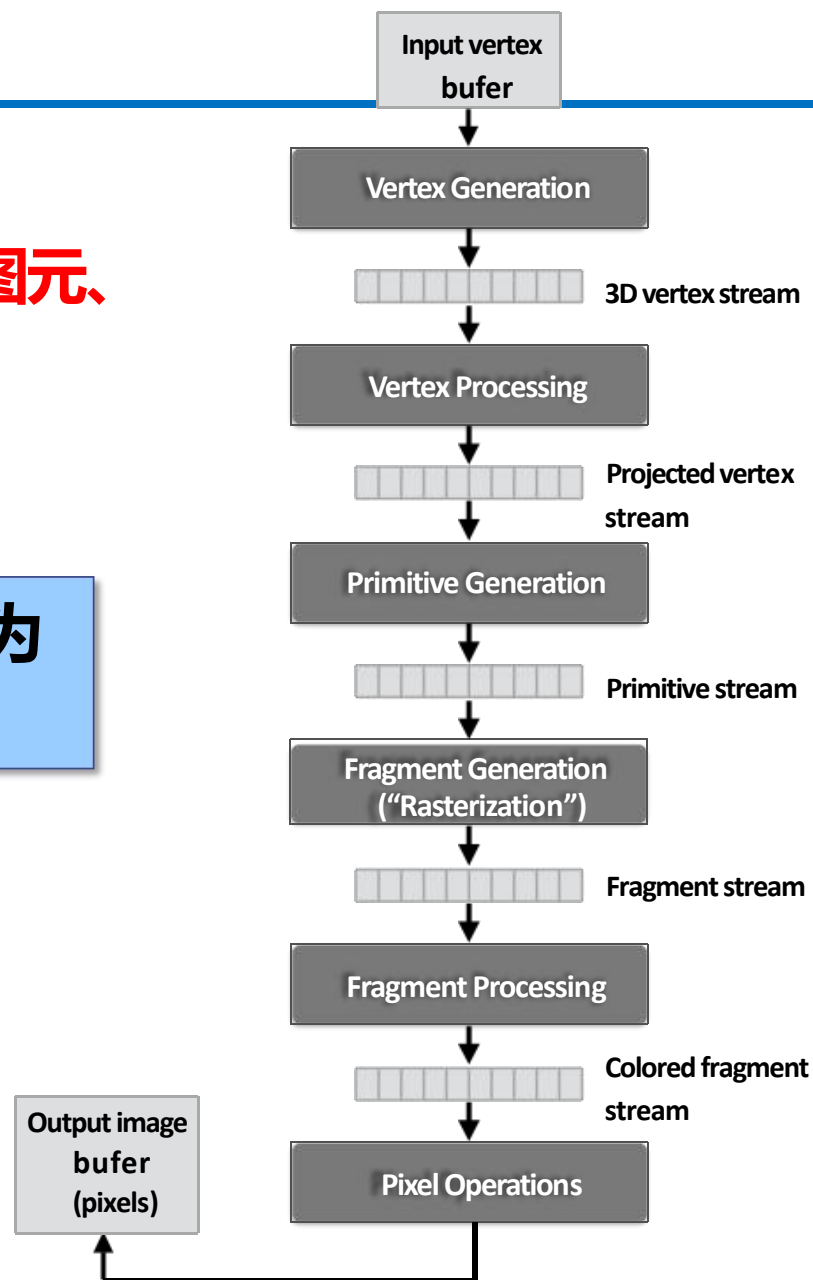
Step 5: 根据片段生成最终像素



渲染管线

将渲染图像的过程抽象为对顶点、图元、片段和像素的一系列操作

渲染过程是高度并行化的，因为像素之间通常没有关联



GPU渲染程序：片段着色

根据纹理图片对片段进行着色 (贴图)

OpenGL程序：

```
uniform sampler2D myTexture; 纹理图片
uniform float3 lightDir; 光线方向
varying vec3 norm; 法线向量
varying vec2 uv; 该片段的纹理坐标

void myFragmentShader()
{
    vec3 kd = texture2D(myTexture, uv);
    kd *= clamp(dot(lightDir, norm), 0.0, 1.0);
    return vec4(kd, 1.0);
}
```

该程序所有片段并行执行

myTexture 纹理中获取在 uv 纹理坐标位置的像素颜色

计算光线方向向量 lightDir 和表面法线 norm 的点积(光线与表面法线的夹角余弦值)

GPU渲染程序：片段着色

根据纹理图片对片段进行着色 (贴图)

OpenGL程序：

```
uniform sampler2D myTexture; 纹理图片
uniform float3 lightDir; 光线方向
varying vec3 norm; 法线向量
varying vec2 uv; 该片段的纹理坐标

void myFragmentShader()
{
    vec3 kd = texture2D(myTexture, uv);
    kd *= clamp(dot(lightDir, norm), 0.0, 1.0);
    return vec4(kd, 1.0);
}
```

该程序所有片段并行执行

myTexture 纹理中获取在 uv 纹理坐标位置的像素颜色

kd 代表漫反射系数或者光照强度的衰减系数

GPU渲染程序：片段着色

根据纹理图片对片段进行着色 (贴图)

OpenGL程序：

```
uniform sampler2D myTexture; 纹理图片
uniform float3 lightDir; 光线方向
varying vec3 norm; 法线向量
varying vec2 uv; 该片段的纹理坐标

void myFragmentShader()
{
    vec3 kd = texture2D(myTexture, uv);
    kd *= clamp(dot(lightDir, norm), 0.0, 1.0);
    return vec4(kd, 1.0);
}
```

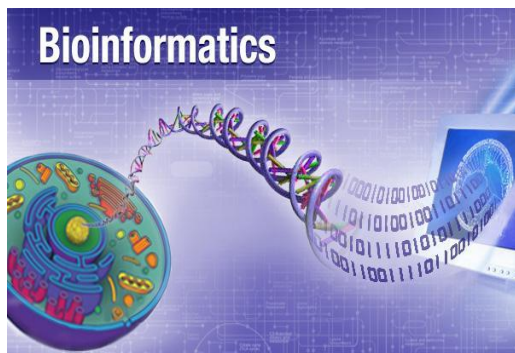
该程序所有片段并行
执行

myTexture 纹理中获取在
uv 纹理坐标位置的像素颜色

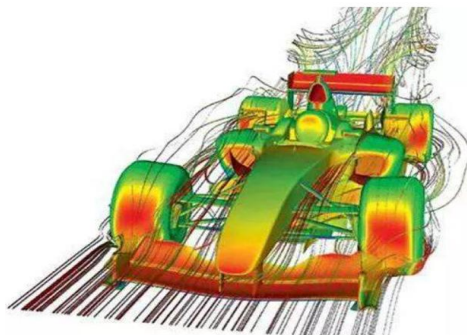
返回着色结果

GPU用于通用计算

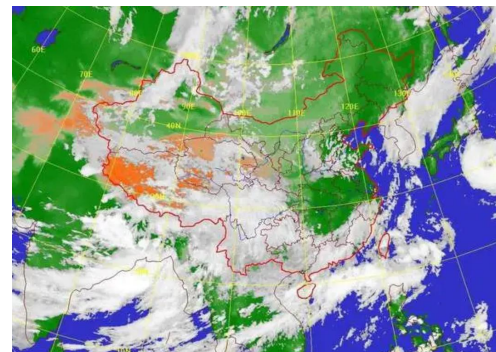
GPU用途广泛



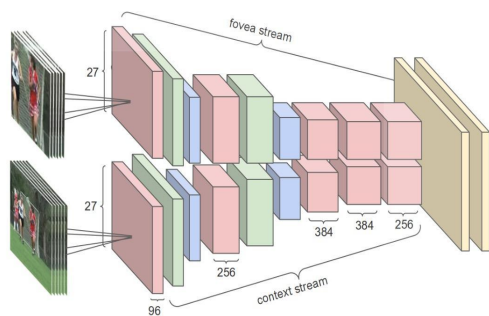
生物计算



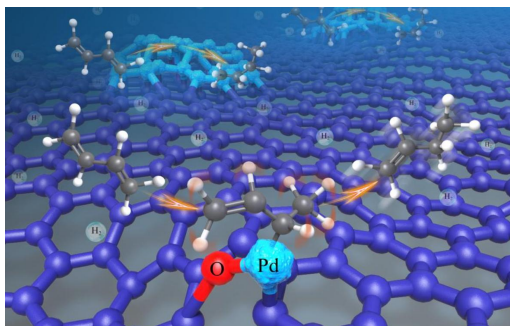
流体仿真



天气预报



深度学习



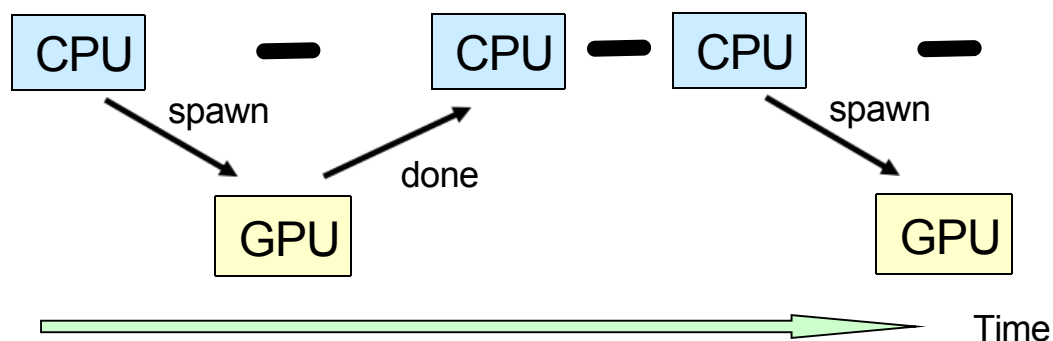
计算化学



计算金融

GPU程序模型

- CPU将并行计算任务卸载到GPU
- 将数据传到GPU memory
- GPU启动多线程完成计算任务
- GPU将计算结果传回CPU



CUDA程序模型

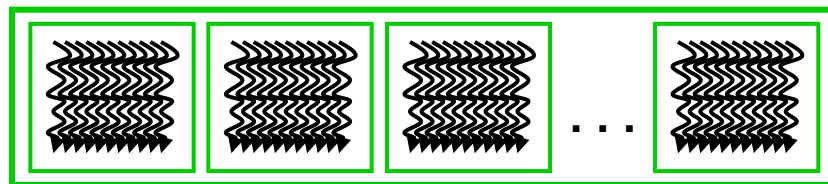
- **host** : 指代CPU端的代码
- **device** : 指代GPU端的代码
- **kernel** : 从host调用, 在device端运行的函数

Serial Code (host)



Parallel Kernel (device)

`KernelA<<< nBlk, nTid >>>(args);`

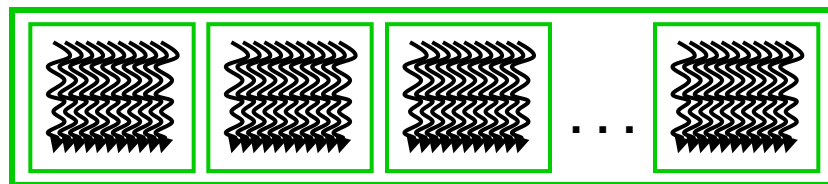


Serial Code (host)



Parallel Kernel (device)

`KernelB<<< nBlk, nTid >>>(args);`



GPU代码示例：矩阵相加

CPU版本

```
void MatAdd(float A[N][N], float B[N][N], float C[N][N]) {  
    for(int i = 0; i < N; i++)  
        for(int j = 0; j < N; j++)  
            C[i][j] = A[i][j] + B[i][j];  
}  
  
int main() {  
    ...  
    MatAdd(A, B, C);  
    ...  
}
```

矩阵相加: CUDA Code

```
// Kernel definition
__global__ void MatAdd(float A[N][N], float B[N][N],
                      float C[N][N]) {
    int i = threadIdx.x;
    int j = threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}

int main() {
    ...
    // Kernel invocation with 1 block of N*N*1 threads
    int numBlocks = 1;
    dim3 threadsPerBlock(N, N);
    MatAdd<<<numBlocks, threadsPerBlock>>>>(A, B, C);
    ...
}
```

计算结果矩阵的元素
(i,j)

矩阵相加: CUDA Code

```
// Kernel definition
__global__ void MatAdd(float A[N][N], float B[N][N],
                      float C[N][N]) {

    int i = threadIdx.x;
    int j = threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}

int main() {
    ...
    // Kernel invocation with 1 block of N*N*1 threads
    int numBlocks = 1;
    dim3 threadsPerBlock(N, N);
    MatAdd<<<numBlocks, threadsPerBlock>>>>(A, B, C);
    ...
}
```

计算结果矩阵的元素 (i,j)

使用 N^2 个线程

矩阵相加: CUDA Code

```
// Kernel definition
__global__ void MatAdd(float A[N][N], float B[N][N],
                      float C[N][N]) {

    int i = threadIdx.x;
    int j = threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}

int main() {
    ...
    // Kernel invocation with 1 block of N*N*1 threads
    int numBlocks = 1;
    dim3 threadsPerBlock(N, N);
    MatAdd<<<numBlocks, threadsPerBlock>>>>(A, B, C);
    ...
}
```

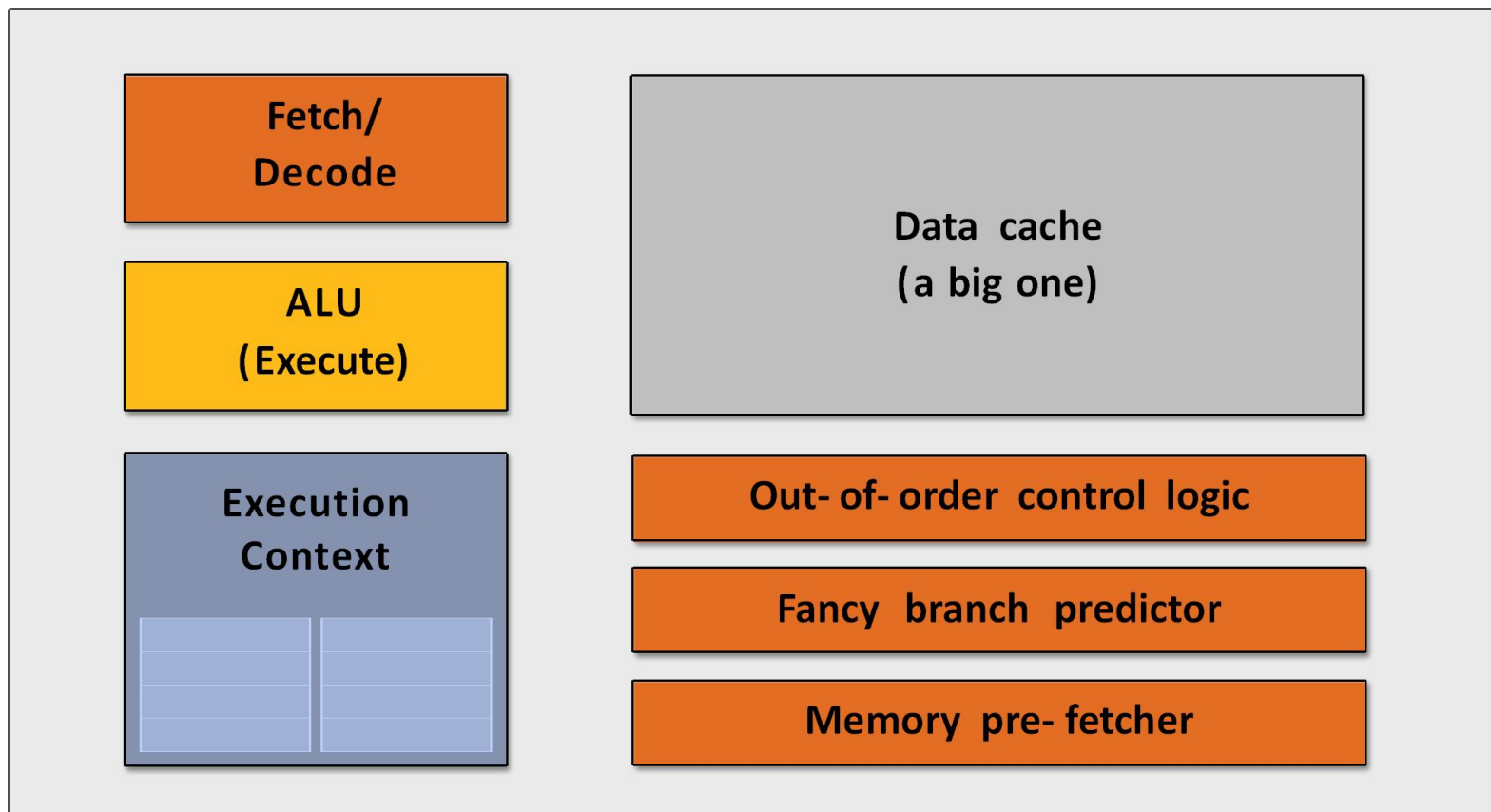
计算结果矩阵的元素 (i,j)

使用 N^2 个线程

启动kernel开始执行

GPU架构

回顾：CPU架构



采用复杂流水线提供指令级并行性！

GPU 设计思路: (1) 简化流水线, 增加核数

Fetch/
Decode

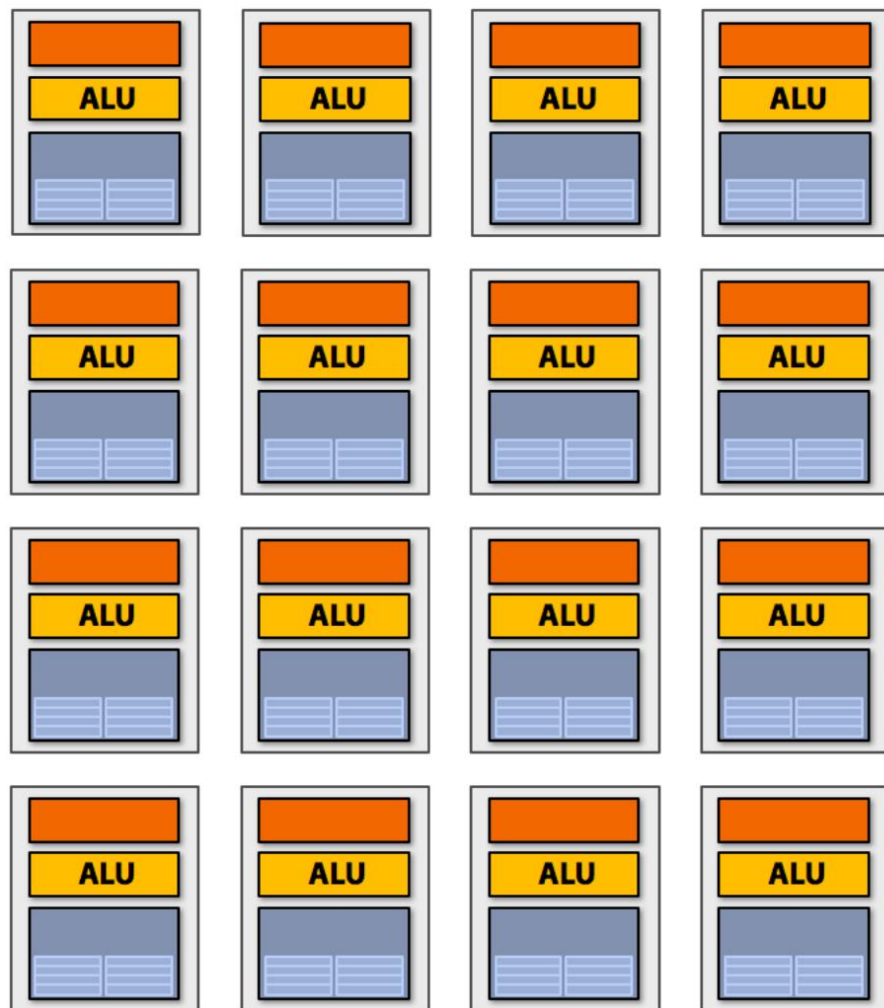
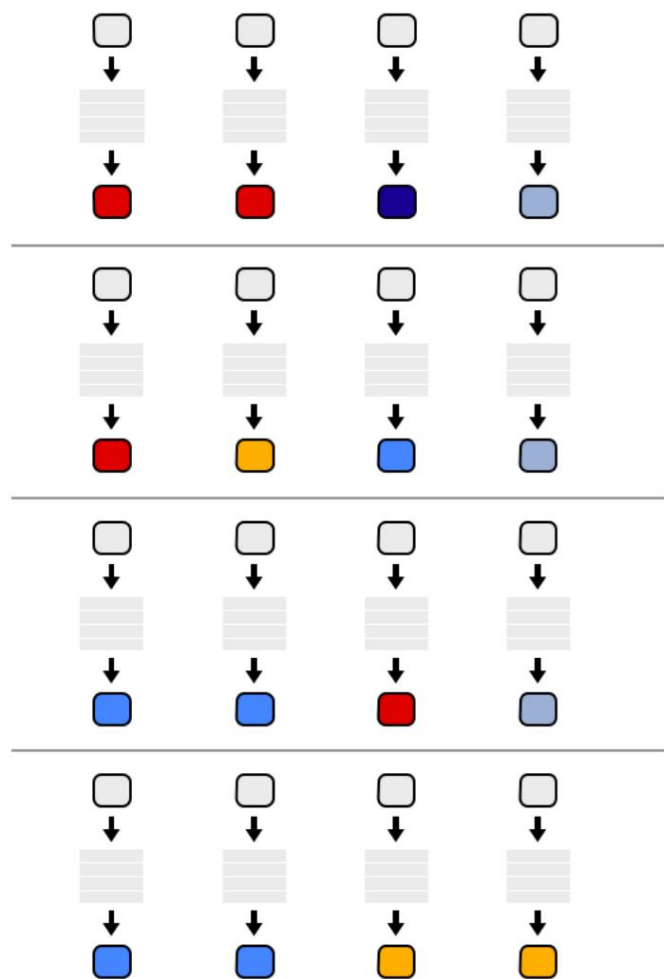
ALU
(Execute)



Idea #1:

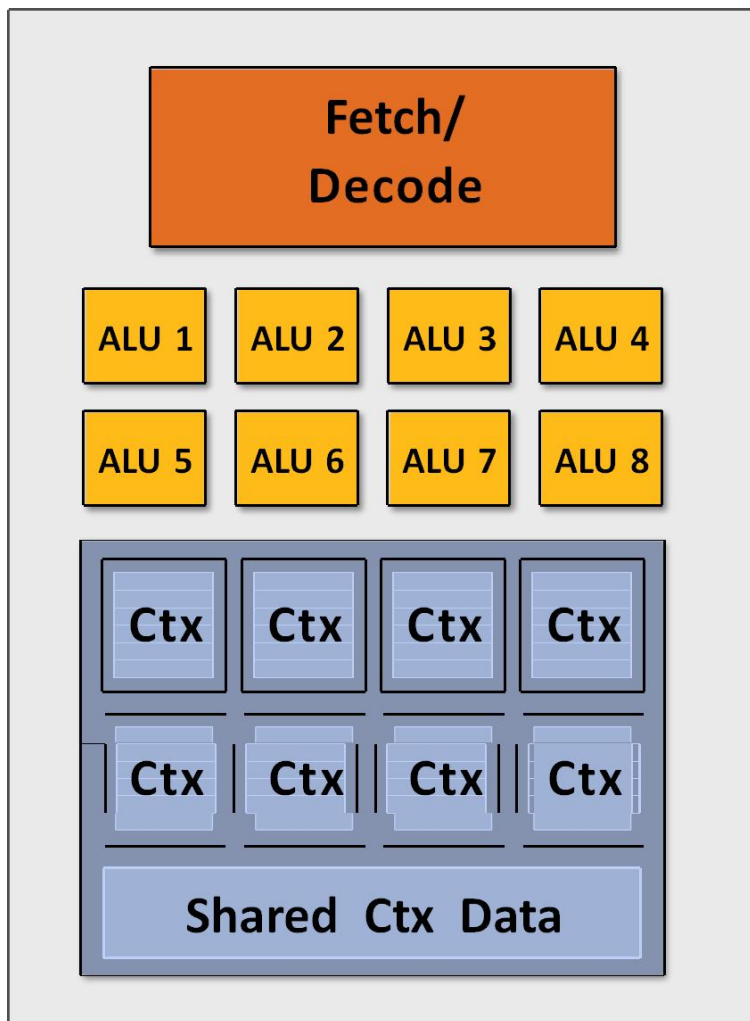
对流水线进行瘦身, 去掉乱序执行、分支预测等复杂逻辑, 节省的空间用来增加大量核心

GPU 设计思路: (1) 简化流水线, 增加核数



节省的晶体管用来增加核心个数，并行处理多个数据

GPU 设计思路: (2) 单指令多线程

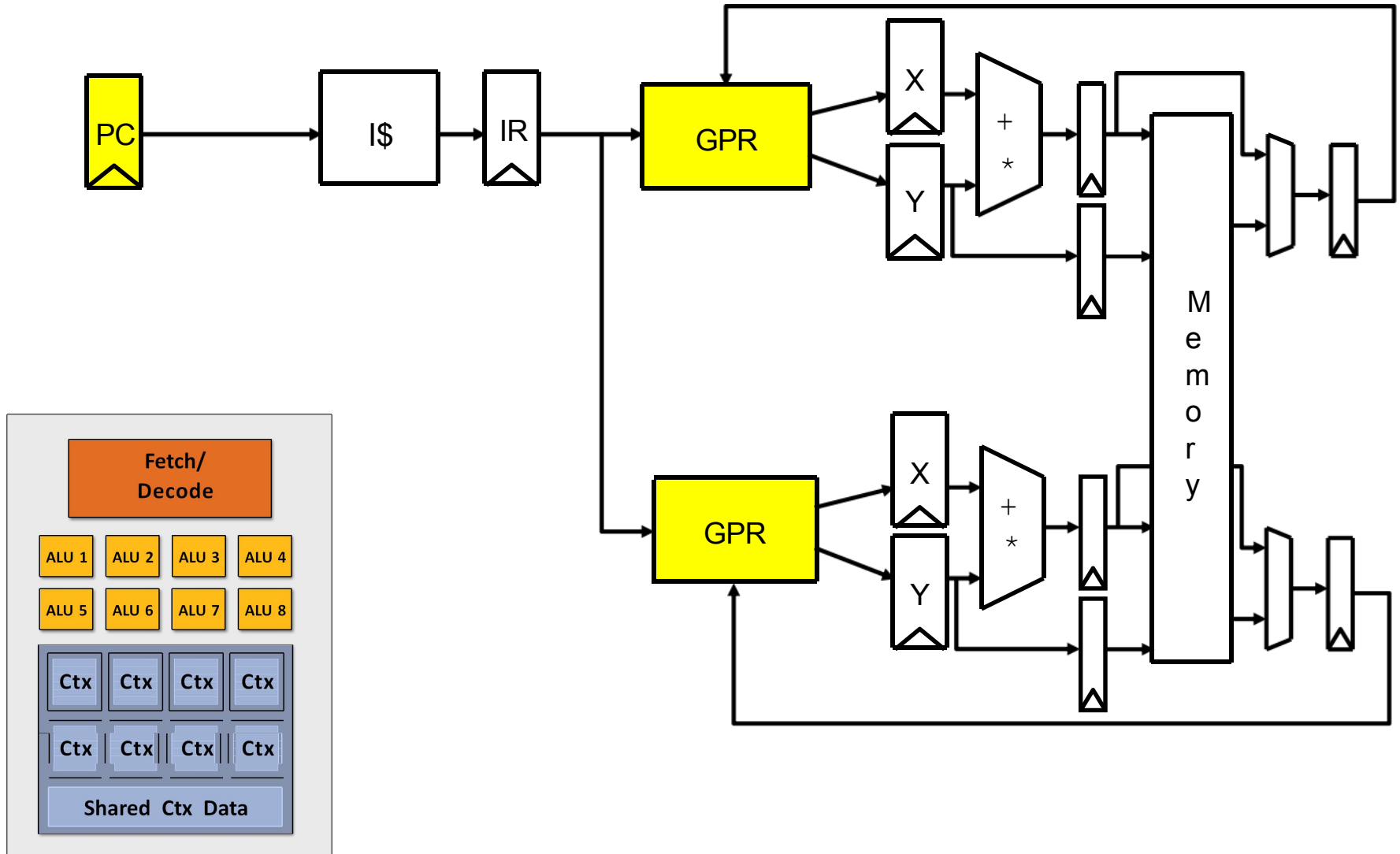


Idea #2:

多个核心共用一条指令, 将管理指令流的成本/复杂性分摊到多个运算单元上(PE)

SIMT(单指令多线程)

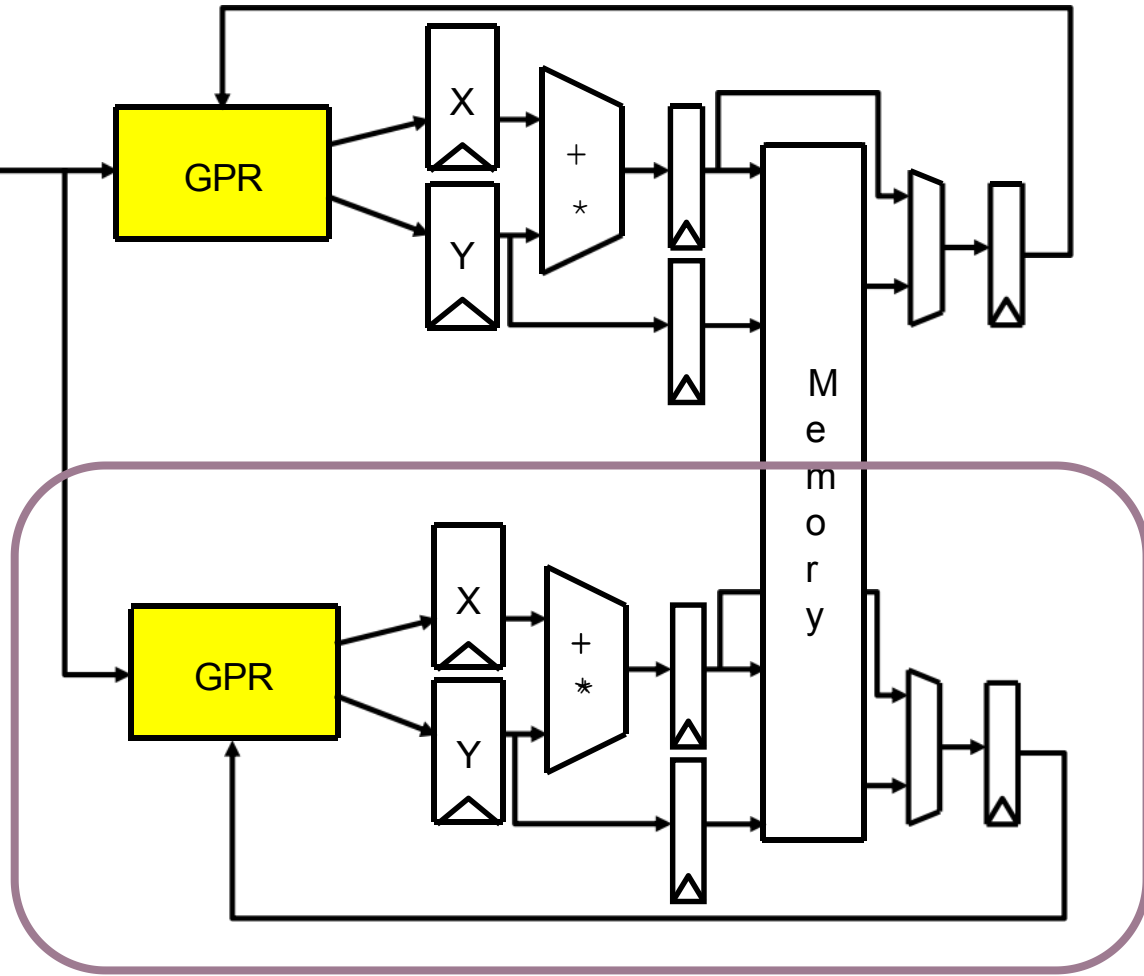
单指令多线程 (Single Instruction Multiple Thread)



单指令多线程 (Single Instruction Multiple Thread)

SIMT

- 多个**threads**, 每个有自己的私有体系结构状态(寄存器)
- 一组共同发射的线程称为一个**warp(Nvidia术语)**
- 所有一块发射的**threads** 执行同一条指令

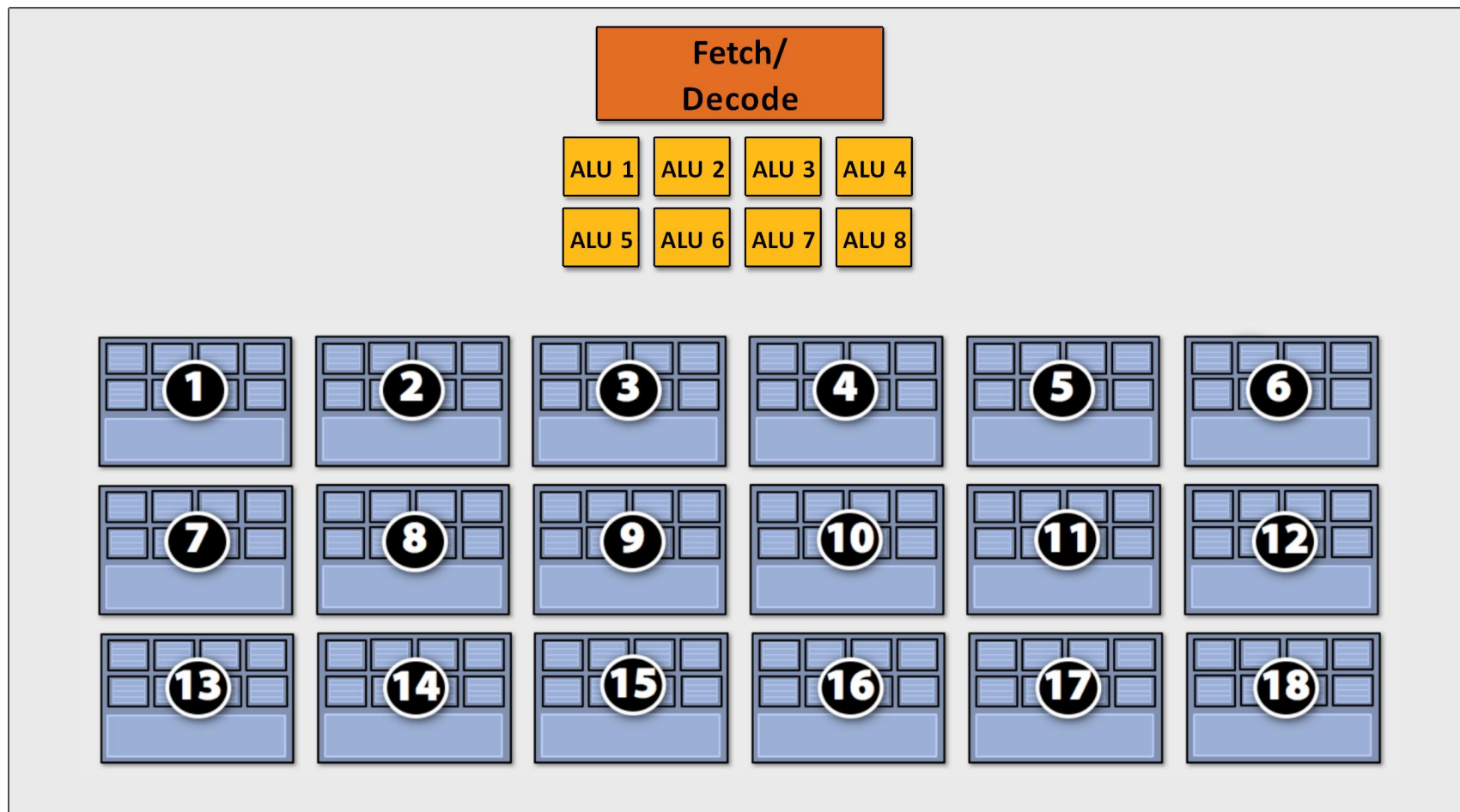


绿色的字是Nvidia的术语

GPU 设计思路: (3) 同时驻留大量线程

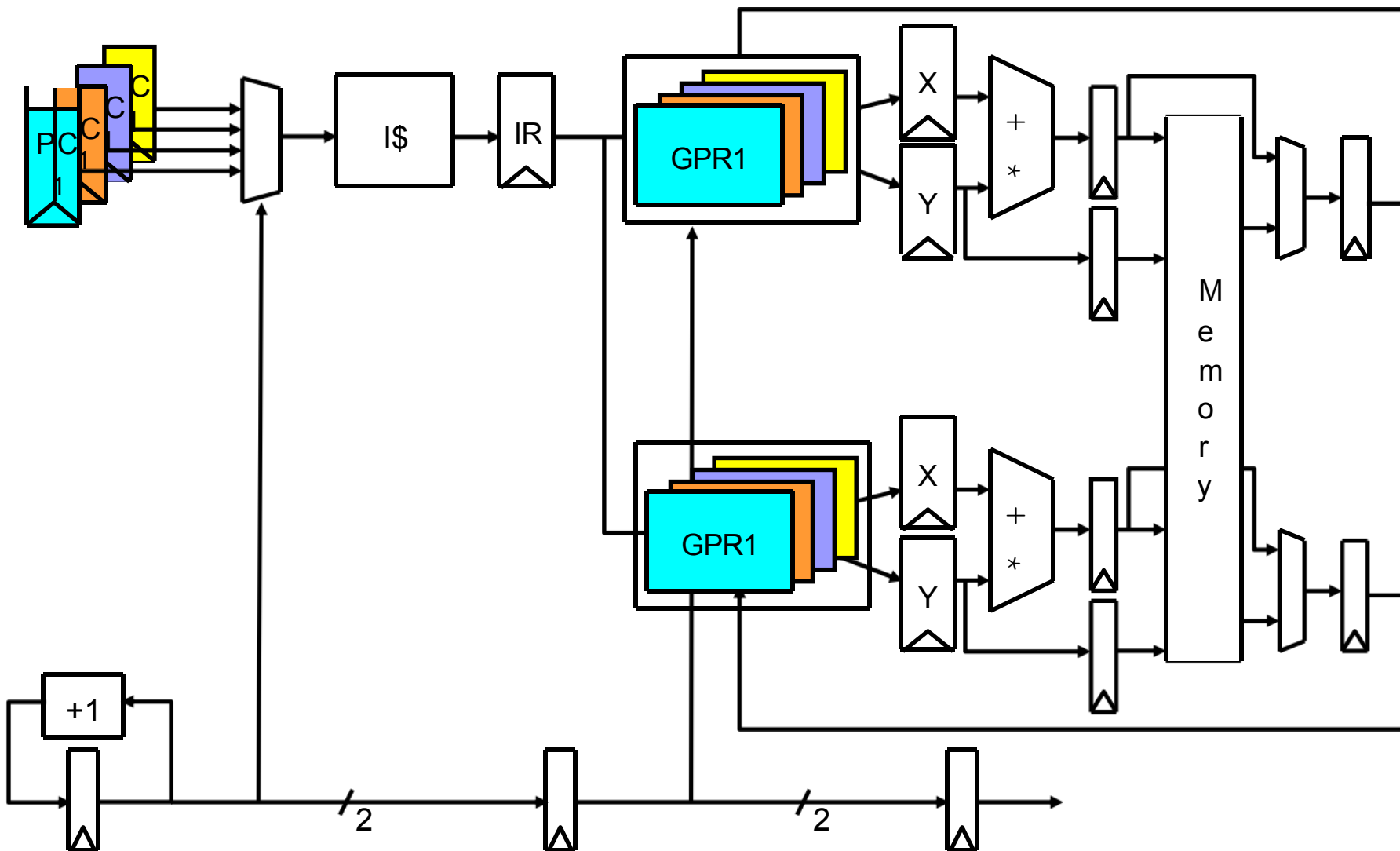
Idea #3:

在单核心上维护远多于执行单元的线程数,以实现细粒度线程调度掩盖高延迟操作



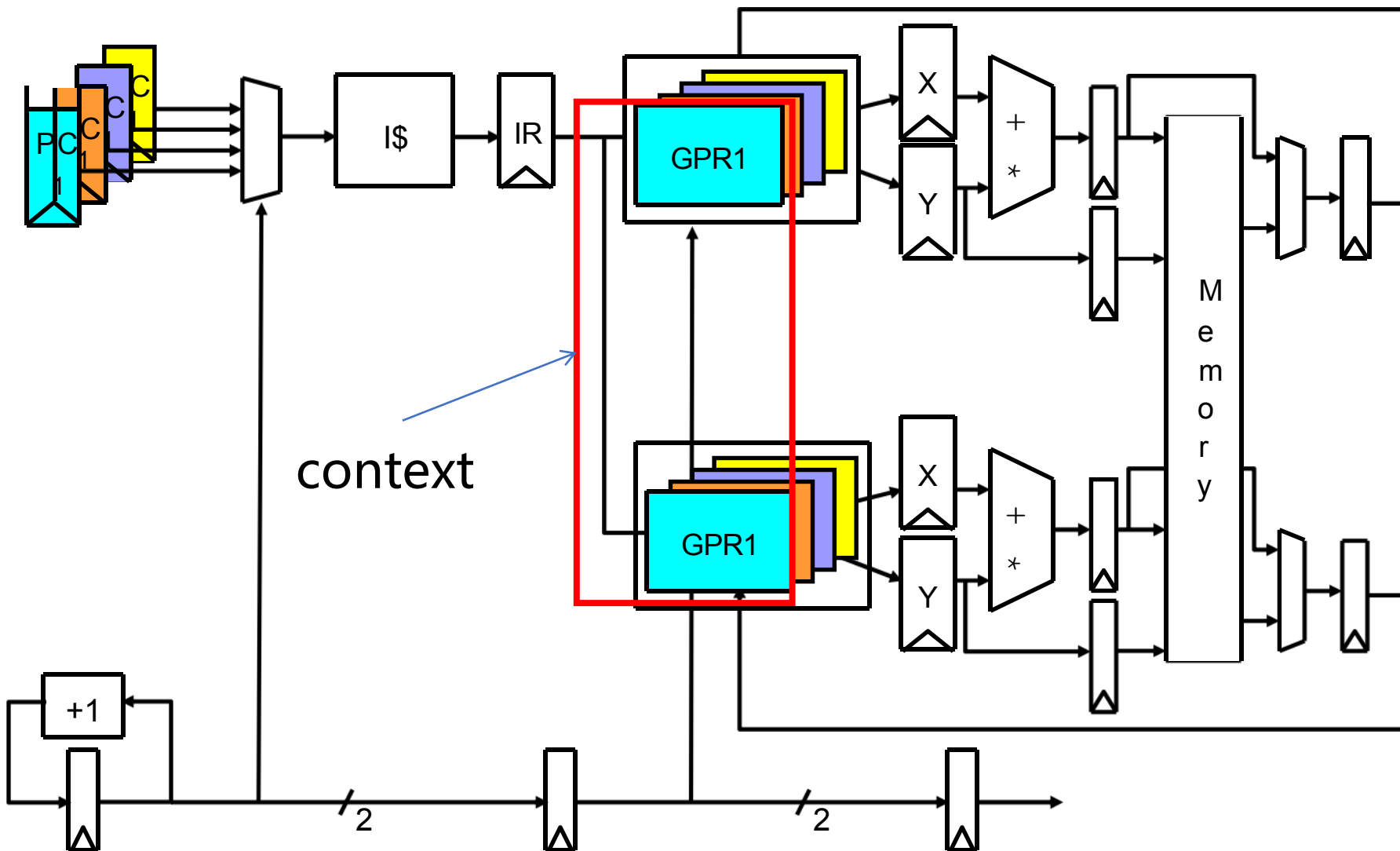
多线程+单指令多线程

(Multithreading + Single Instruction Multiple Thread)



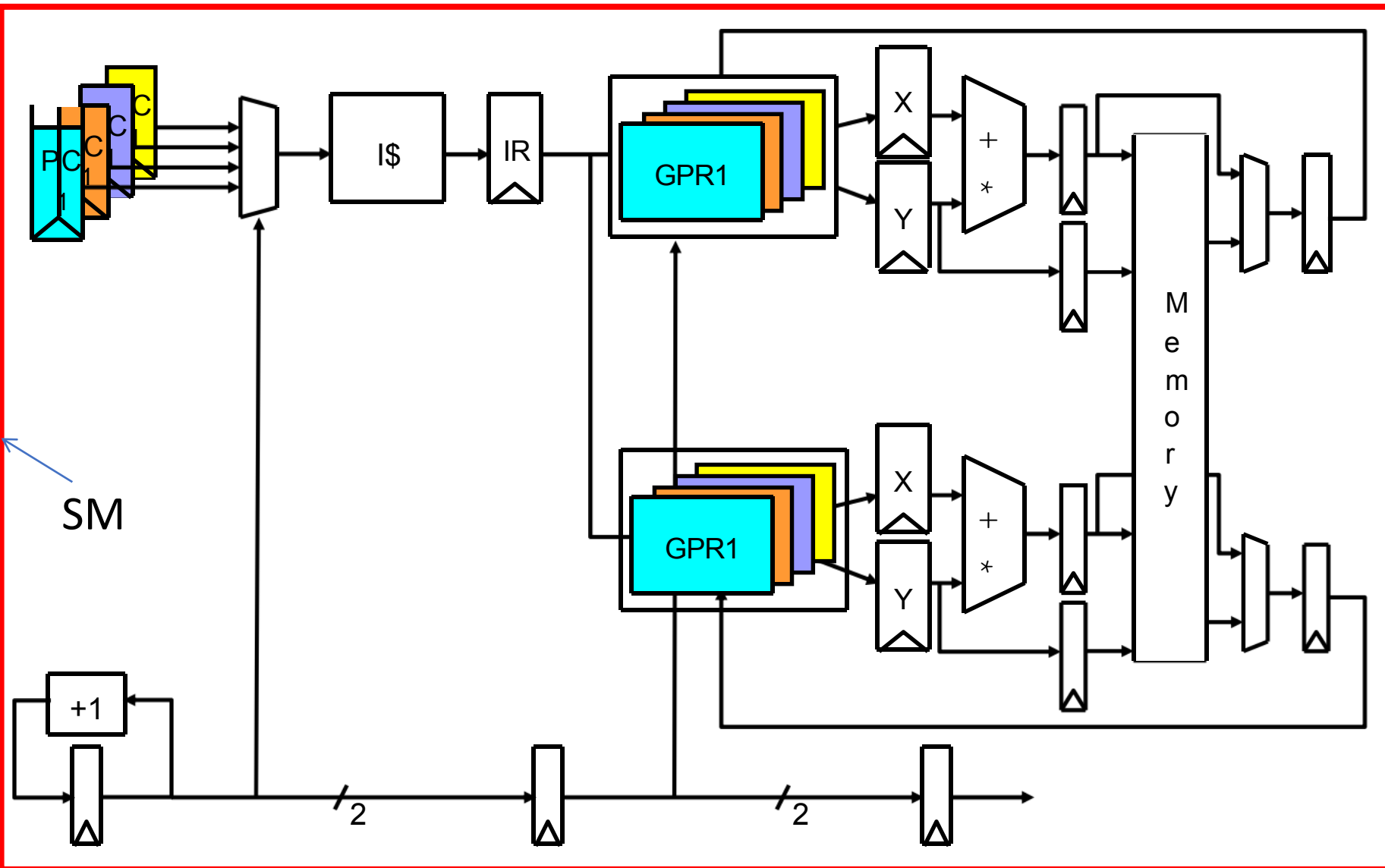
多线程+单指令多线程

(Multithreading + Single Instruction Multiple Thread)

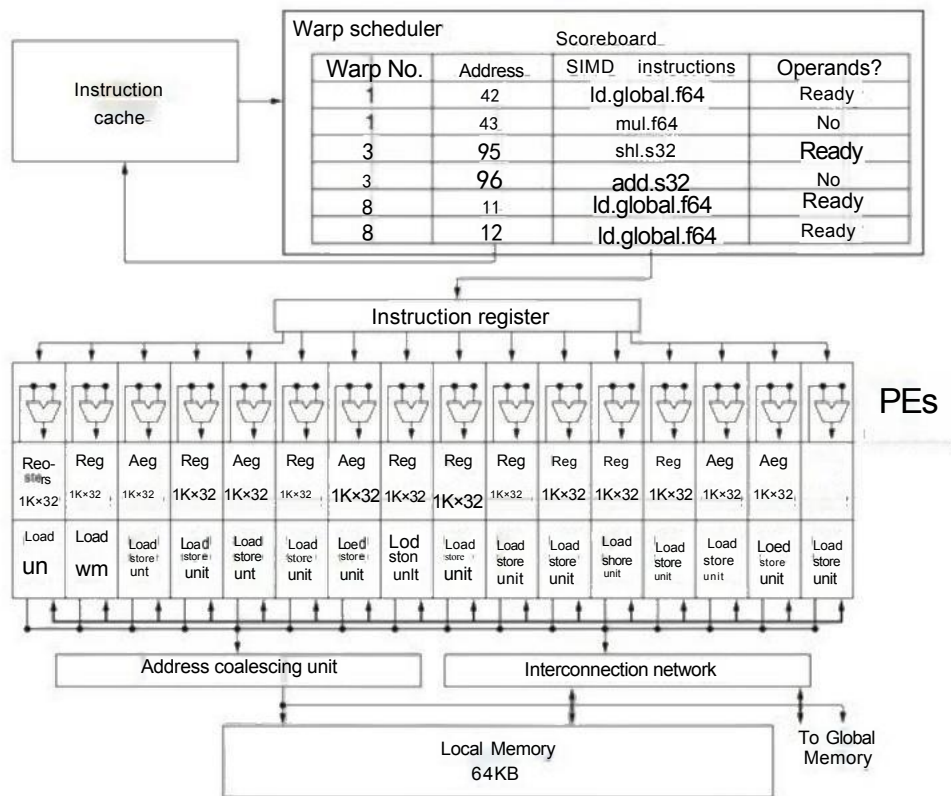


多线程+单指令多线程

(Multithreading + Single Instruction Multiple Thread)



SM整体结构



- 每个SM支持几十个warps (e.g., 64 in Kepler), 每个warp 32个线程
- 每个周期取1条指令(来自某个warp)
- 每个周期发射1条指令, 简单记分板处理相关性
- 指令广播到每个PE
- 多线程是掩盖延迟的主要技术

GPU整体架构：Nvidia Fermi

包含16个SM，每个SM是一个垂直的矩形条带

每个SM有32个CUDA内核(PE)，共计512个

Giga Thread全局调度器将线程块分发给SM线程调度器

一个CUDA内核为一个线程在每个时钟周期里执行一条浮点或整数指令



GPU整体架构：Nvidia Fermi

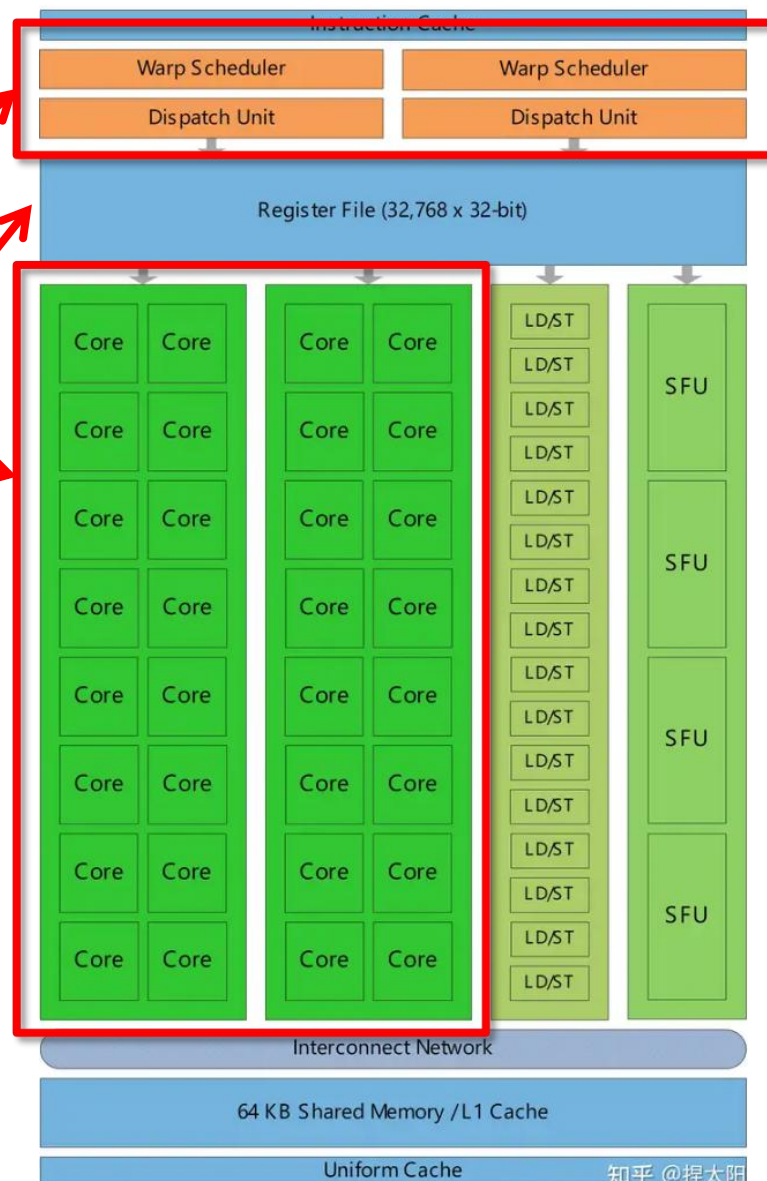
右图为SM内部结构

2个Warp Scheduler负责SM内部warp调度

绿色部分：32个CUDA内核(PE),分为两组, 每组16个

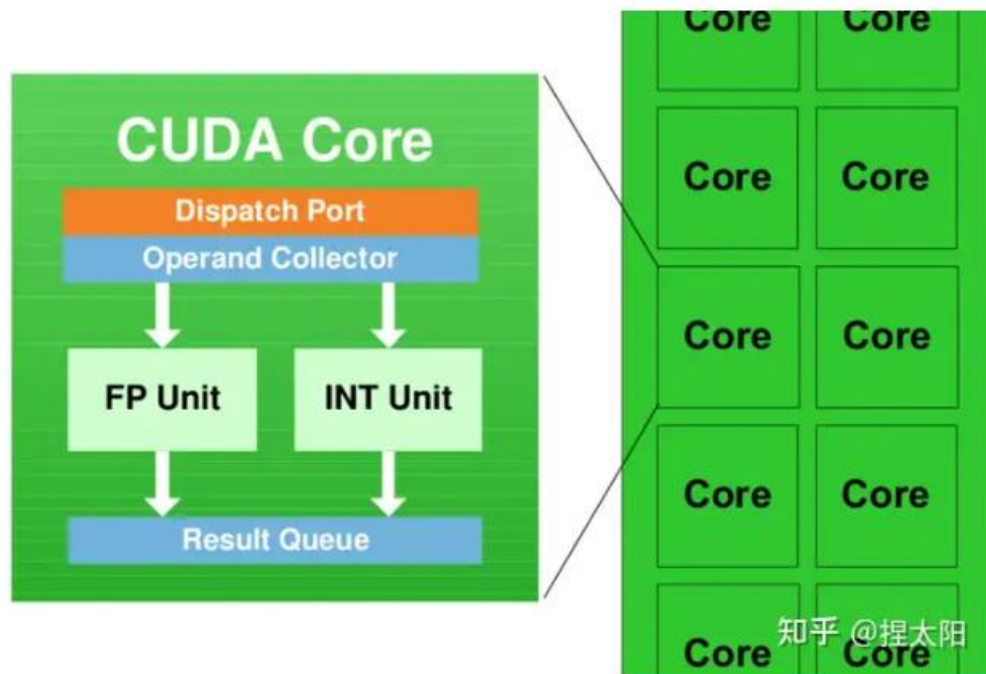
浅蓝色部分：寄存器文件和L1 cache, 访存单元等

共包含32768个寄存器，最多支持512个线程驻留(512个context)



GPU整体架构：Nvidia Fermi

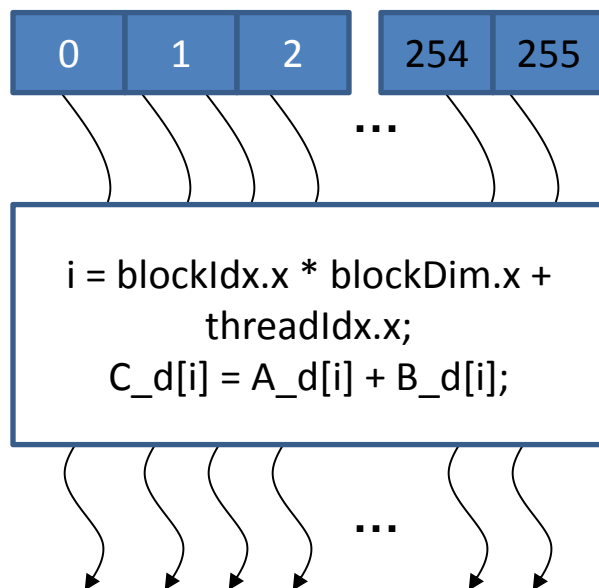
右图为**CUDA内核(PE)**内部结构，每个 CUDA 内核都有一个**完全流水线化的整数算术逻辑单元 (ALU)** 和**浮点单元 (FPU)**



GPU 线程调度

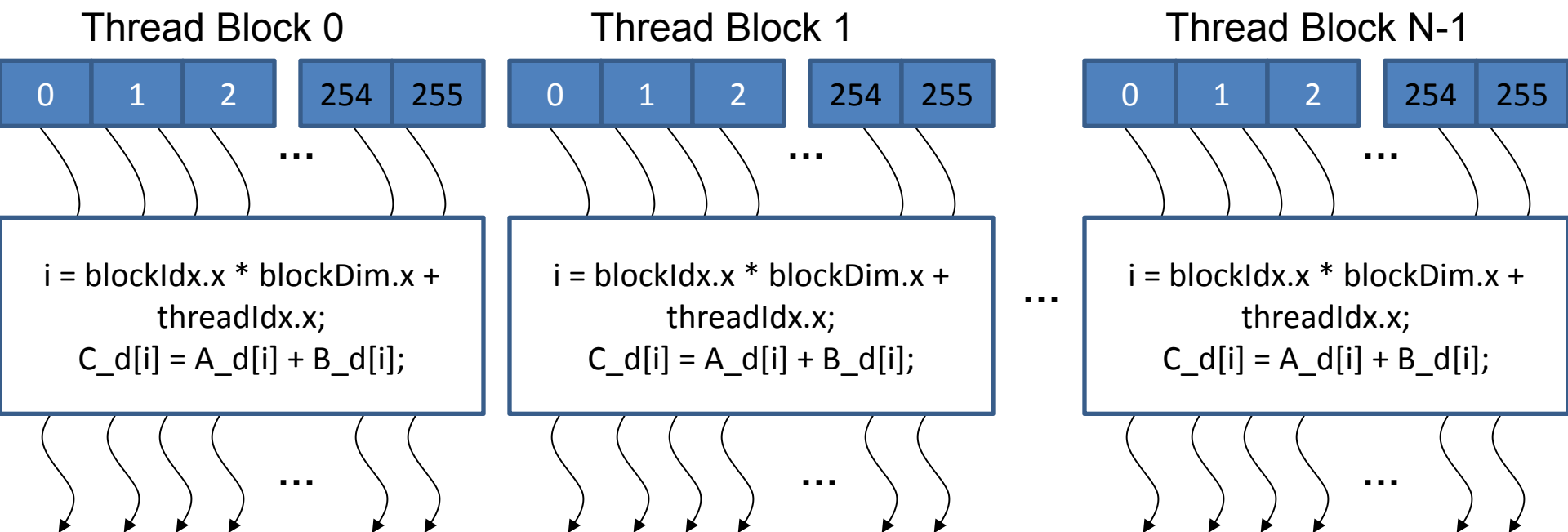
GPU 线程

- 一个GPU程序对应一个**device kernel**
- Kernel以一组线程的方式执行(**称为Grid**)
- 所有线程执行相同的kernel代码
- 每个线程使用自己的编号，计算不同数据



线程块 (Block)

- 一个Grid的所有线程分成若干块(**blocks**)
- **Block内线程协同计算**：共享内存、原子操作、同步机制
- Block间不能协作



线程和Block编号

- Block和thread采用**多维编号**(便于映射到数据)

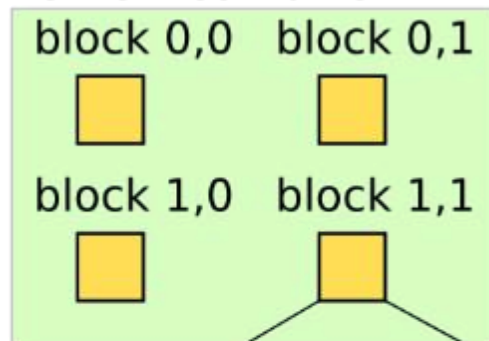
Kernel包含2x2个
blocks 每个block包含
 2x4个threads

```
kernelF<<<(2,2),(2,4)>>>(A);
```

```
__device__ kernelF(A){  
    i = blockDim.x * blockIdx.x  
      + threadIdx.x;  
    j = blockDim.y * blockIdx.y  
      + threadIdx.y;  
    A[i][j]++;  
}
```

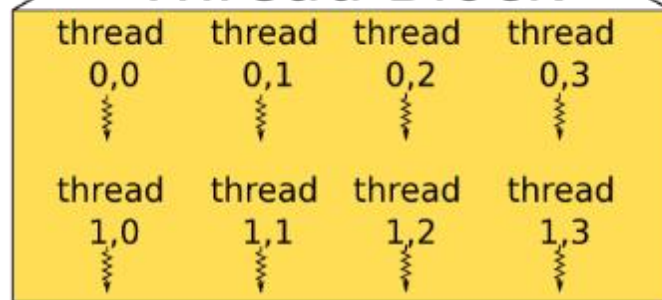
Grid

kernelF contains 2 x 2 thread blocks



Thread ↕

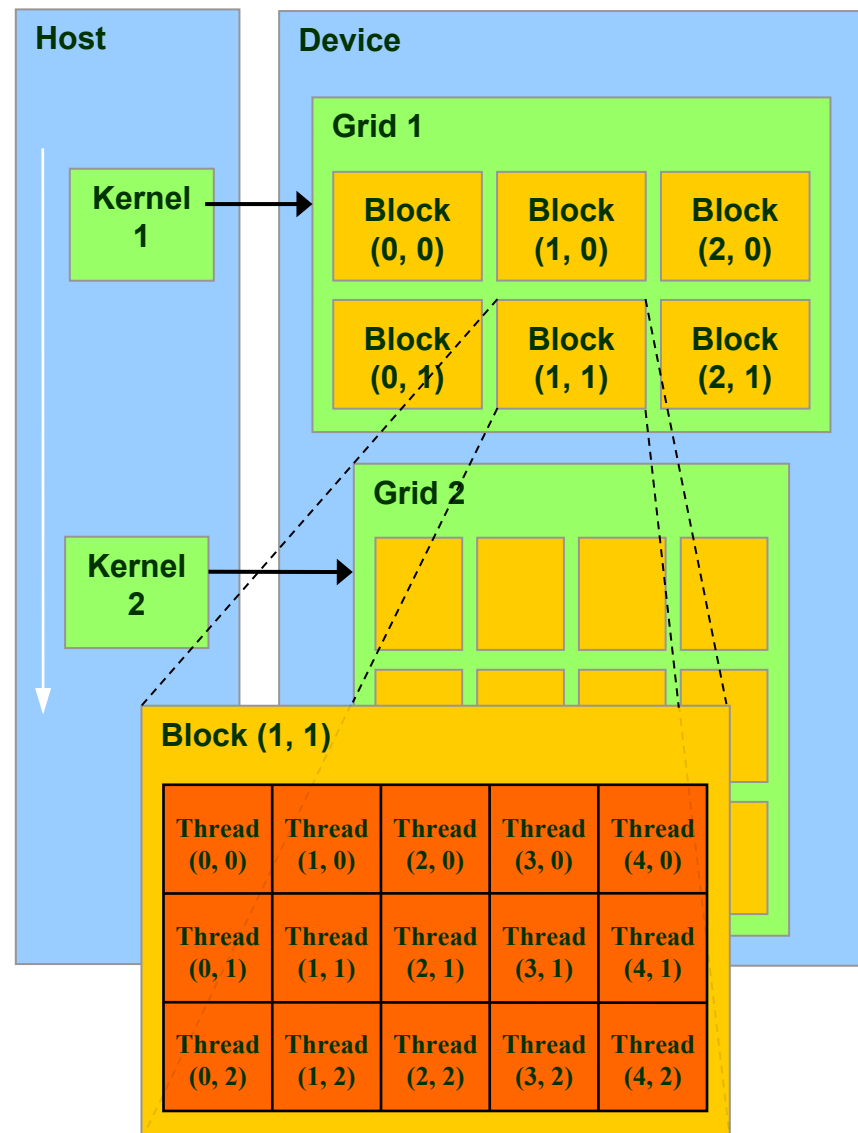
Thread Block



Each thread block contains 2 × 4 threads

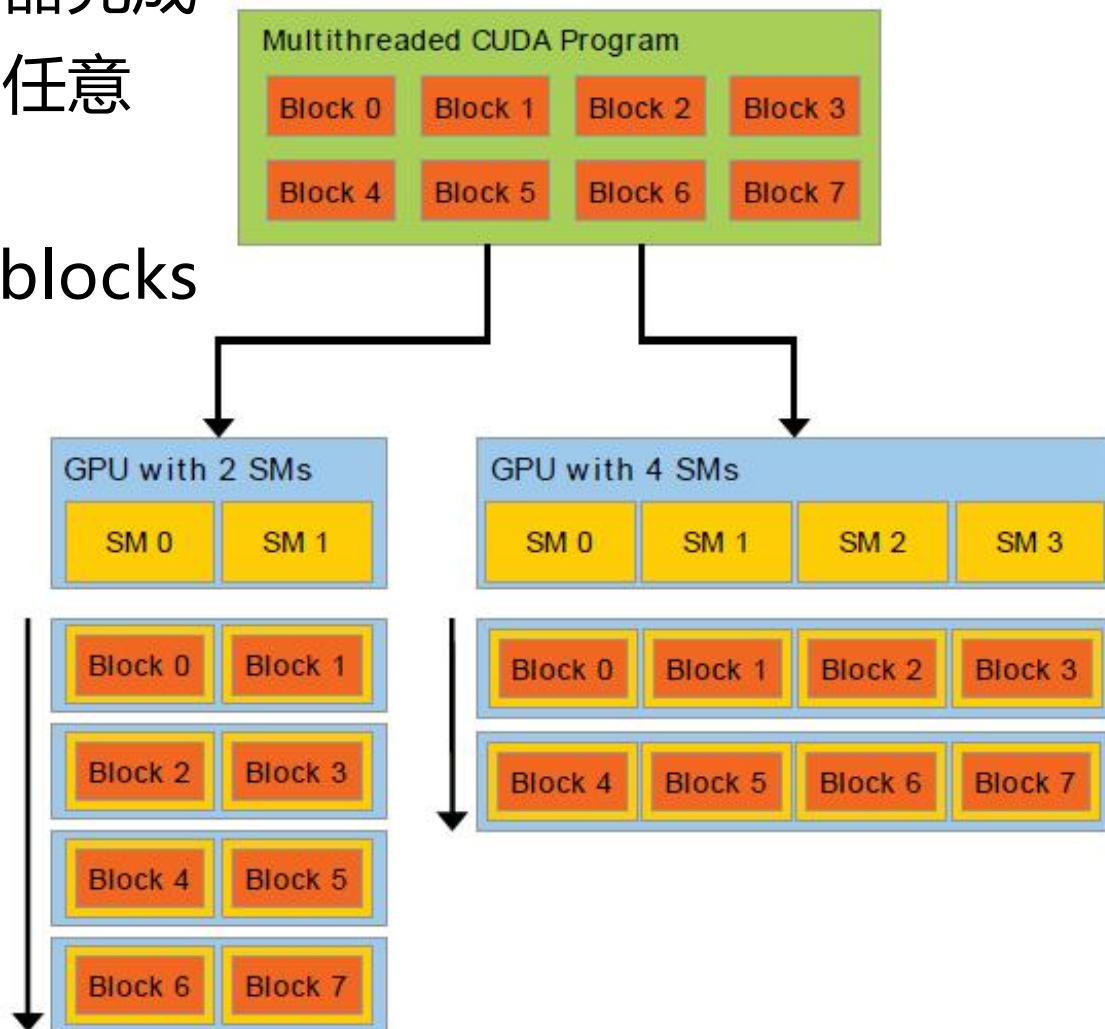
线程在GPU上的生命周期

- 首先, 所有线程(Grid)被启动
- Thread Blocks被调度到SM
- 每个SM可能处理多个 blocks
- 每个SM以Warp为粒度调度线程
- 所有Warps和Blocks执行完, 释放资源



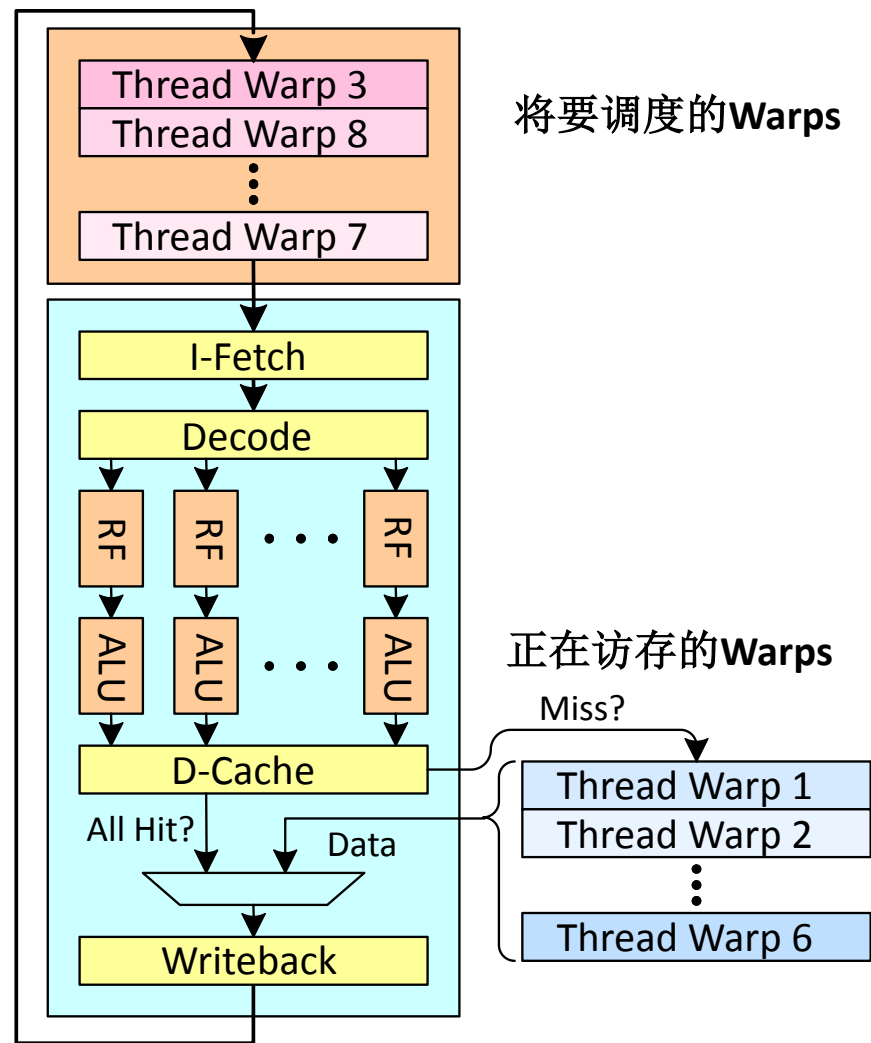
Block 调度

- block调度由硬件调度器完成
- 每个block可以调度到任意SM上
- 每个SM上可驻留多个blocks
- 多组context



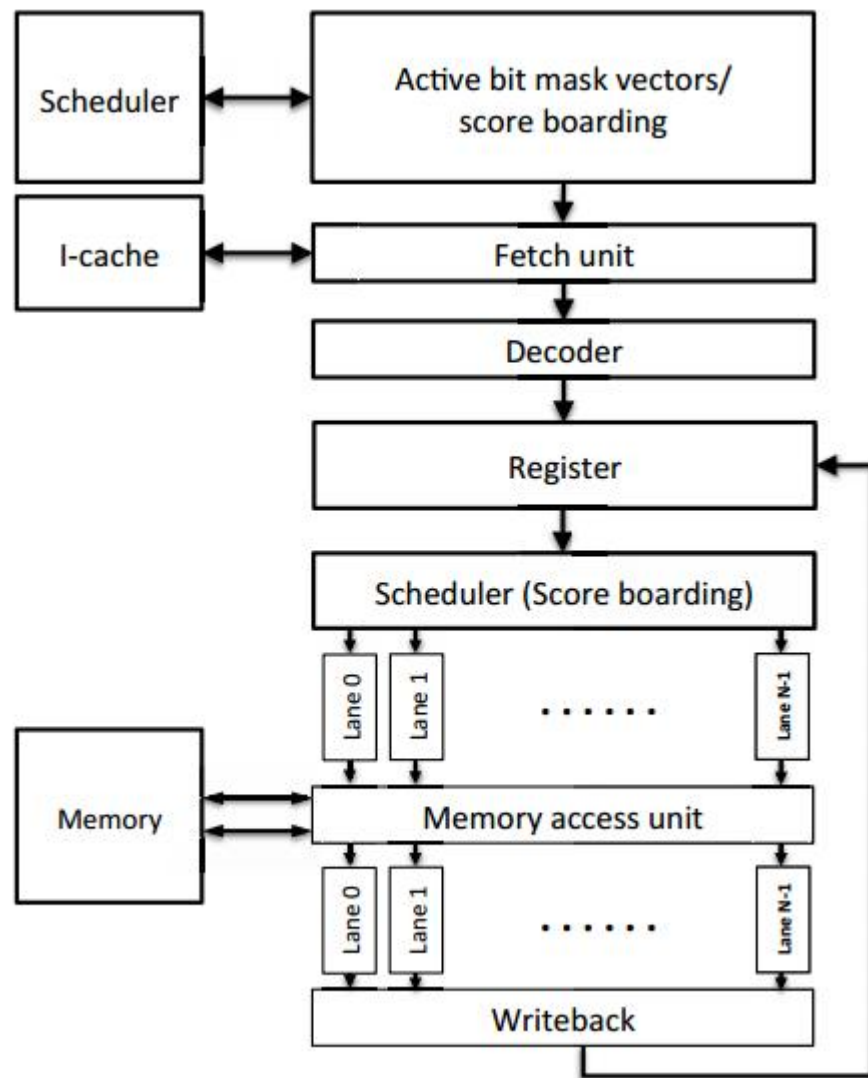
线程调度

- block调度到SM之后，以warp为单位进行调度执行
- 一个block被划分为多个warp
- 每个warp中的线程并发执行，运行同一条指令，但处理不同的数据
- 多个warps交替执行(每个cycle选一个warp)
 - ✓ fine-grained multithreading，但以warp为单位



SM 指令执行流程

- 每个周期取一个warp的指令
 - 从指令cache
 - 放入指令缓存
- 发射一条可以执行的warp指令
 - 指令的操作数都准备好了
 - 用**记分板**解决数据危害
- 发射指令选择采用轮循方式
- SM将指令广播给所有PE来执行



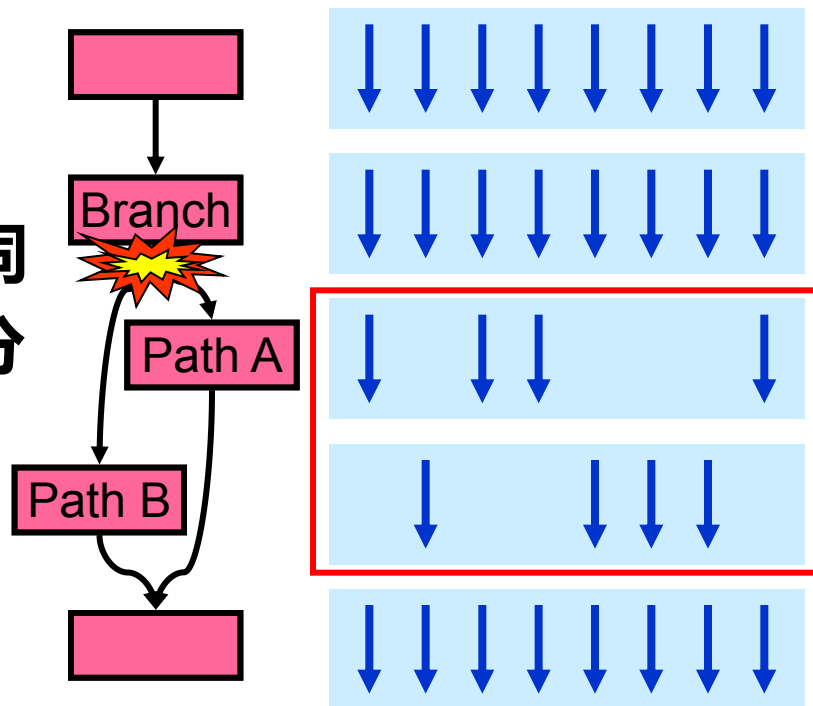
GPU架构关键问题

GPU的控制流问题

□ GPU采用SIMD流水线以节省控制逻辑的面积

- 将线程分成组(warps)

□ 当warps内的线程分支到不同的执行路径时，会发生分支分歧 (**branch divergence**)

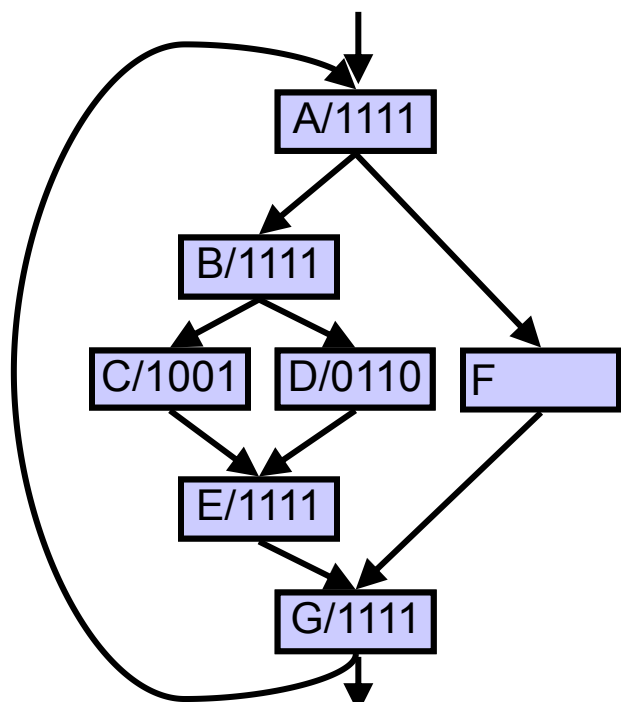


本来warps内的线程执行相同的指令，分支分歧后线程执行的指令不一样！

处理分支分歧

- **每个warp用stack存储不跳转分支的PCs和掩码**
- **遇到分支时**
 - 将当前掩码入栈
 - 将不跳转分支的掩码和PC入栈
 - 设置当前掩码为跳转分支的掩码
- **分支执行完时**
 - 将不跳转分支的掩码和PC出栈，并执行分支
- **不跳转分支执行完时**
 - 将分支之前的源掩码弹出
- 如果一个分支的掩码都是0，跳过该代码块

分支分歧解决方案



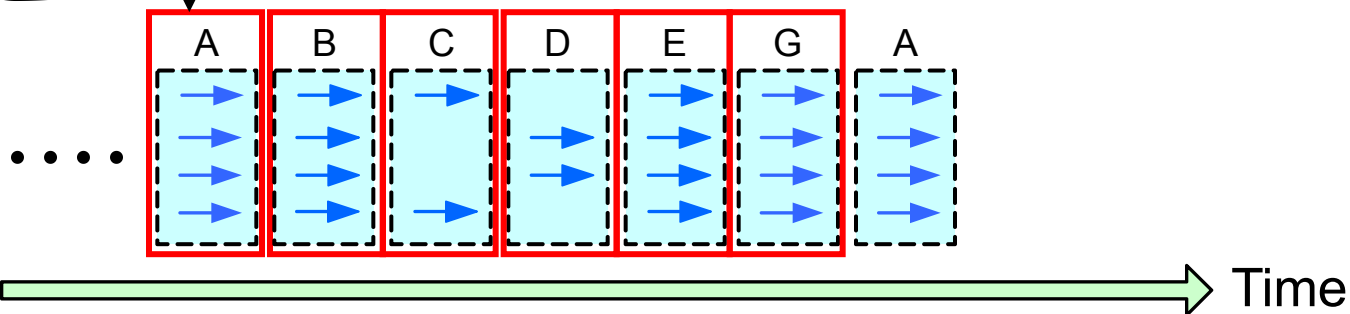
Stack

Reconv. PC	Next PC	Active Mask
-	E	1111
E	D	0110
E	C	1001

Thread Warp

Common PC

Thread 1	Thread 2	Thread 3	Thread 4



Example: 分支分歧

假设 4 threads/warp,
初始掩码 1111

```
1111
    if (m[i] != 0) {
1100    if (a[i] > b[i]) {
1000        y[i] = a[i] - b[i];
        } else {
0100        y[i] = b[i] - a[i];
        }
    } else {
0011    y[i] = 0;
    }
```

Example: 分支分歧

假设 4 threads/warp,
初始掩码 1111

```
1111
if (m[i] != 0) {
1100   if (a[i] > b[i]) {
1000     y[i] = a[i] - b[i];
        } else {
0100     y[i] = b[i] - a[i];
        }
    } else {
0011   y[i] = 0;
    }
```

①

- ① Push mask 1111
Push mask 0011
Set mask 1100

1111
0011

Example: 分支分歧

假设 4 threads/warp,
初始掩码 1111

```
1111
    if (m[i] != 0) {           ①
1100    if (a[i] > b[i]) {     ②
1000        y[i] = a[i] - b[i];
        } else {
0100        y[i] = b[i] - a[i];
        }
    } else {
0011    y[i] = 0;
    }
```

- ① Push mask 1111
Push mask 0011
Set mask 1100
- ② Push mask 1100
Push mask 0100
Set mask 1000

1111
0011
1100
0100

Example: 分支分歧

假设 4 threads/warp,
初始掩码 1111

1111

```
if (m[i] != 0) {
```

①

1100

```
if (a[i] > b[i]) {
```

②

1000

```
    y[i] = a[i] - b[i];
```

```
    } else {
```

0100

```
    y[i] = b[i] - a[i];
```

```
    }
```

```
    } else {
```

0011

```
    y[i] = 0;
```

```
}
```

① Push mask 1111

Push mask 0011

Set mask 1100

② Push mask 1100

Push mask 0100

Set mask 1000

1111
0011
1100
0100

Example: 分支分歧

假设 4 threads/warp,
初始掩码 1111

```
1111
    if (m[i] != 0) {
1100     if (a[i] > b[i]) {
1000         y[i] = a[i] - b[i];
        } else {
0100         y[i] = b[i] - a[i];
        }
    } else {
0011     y[i] = 0;
    }
```

①

②

③

① Push mask 1111
Push mask 0011
Set mask 1100

② Push mask 1100
Push mask 0100
Set mask 1000

③ Pop mask 0100

1111
0011
1100

Example: 分支分歧

假设 4 threads/warp,
初始掩码 1111

```
1111
    if (m[i] != 0) {
1100    if (a[i] > b[i]) {
1000        y[i] = a[i] - b[i];
        } else {
0100        y[i] = b[i] - a[i];
        }
    } else {
0011    y[i] = 0;
    }
```

①

②

③

④

1111
0011

① Push mask 1111
Push mask 0011
Set mask 1100

② Push mask 1100
Push mask 0100
Set mask 1000

③ Pop mask 0100

④ Pop mask 1100

Example: 分支分歧

假设 4 threads/warp,
初始掩码 1111

```
1111
    if (m[i] != 0) {
1100    if (a[i] > b[i]) {
1000        y[i] = a[i] - b[i];
        } else {
0100        y[i] = b[i] - a[i];
        }
    } else {
0011    y[i] = 0;
    }
```

①

②

③

④

⑤

1111

① Push mask 1111
Push mask 0011
Set mask 1100

② Push mask 1100
Push mask 0100
Set mask 1000

③ Pop mask 0100

④ Pop mask 1100

⑤ Pop mask 0011

Example: 分支分歧

假设 4 threads/warp,
初始掩码 1111

```
1111
    if (m[i] != 0) {
1100    if (a[i] > b[i]) {
1000        y[i] = a[i] - b[i];
        } else {
0100        y[i] = b[i] - a[i];
        }
    } else {
0011    y[i] = 0;
    }
```

①

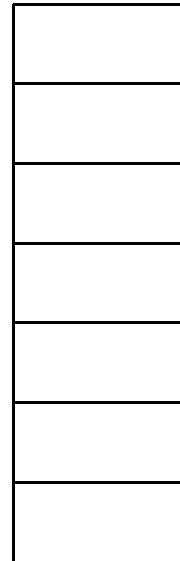
②

③

④

⑤

⑥



① Push mask 1111
Push mask 0011
Set mask 1100

② Push mask 1100
Push mask 0100
Set mask 1000

③ Pop mask 0100

④ Pop mask 1100

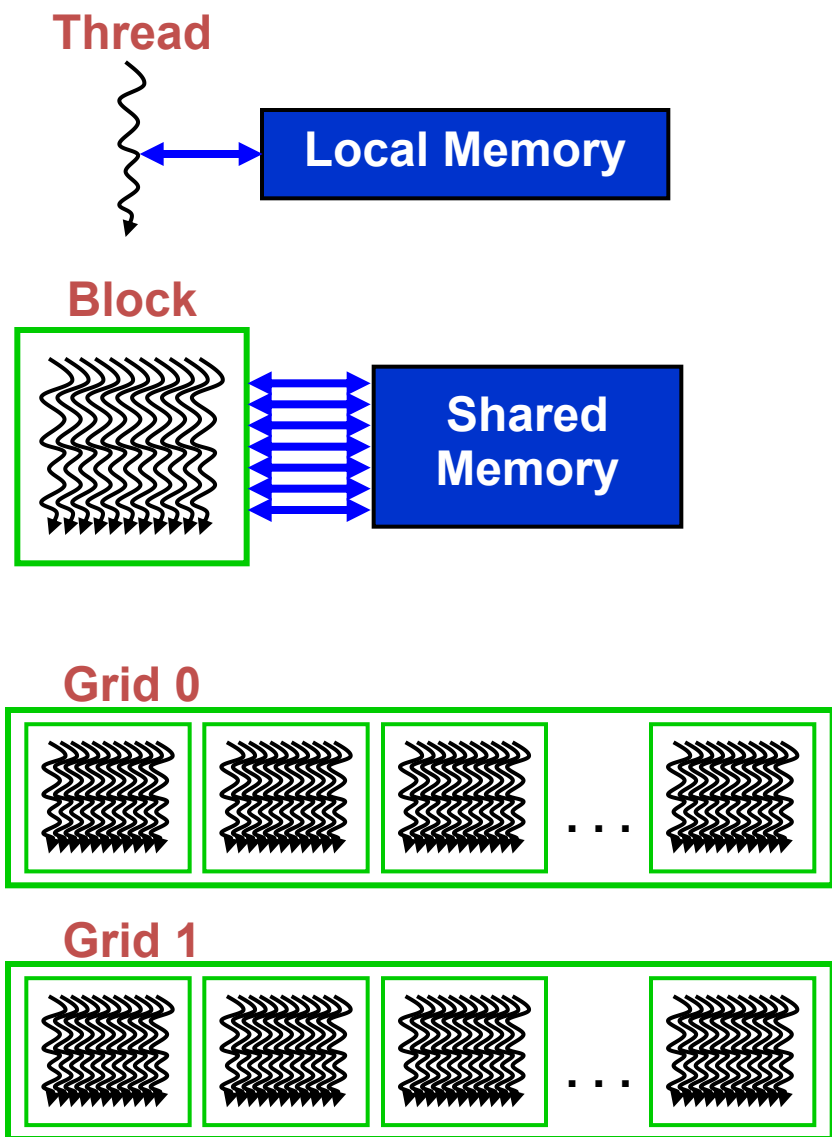
⑤ Pop mask 0011

⑥ Pop mask 1111

访存分歧(Memory Divergence)?

- 现代GPU具有缓存，理想情况下希望warp中的所有线程都能命中缓存（而不会互相冲突）
- **问题：**如果warp中的一个线程在缓存中未命中,可能会导致整个warp停顿
- **需要解决**
 - 容忍访存分歧
 - 整合解决分支和内存分歧的方案

GPU内存结构



- **Local Memory:**
 - 每个线程私有
 - Auto variables, register spill
- **Shared Memory:**
 - 每个SM有一个
 - 同一个block内的线程共享
- **Global Memory: 全局内存**
 - 所有线程共享

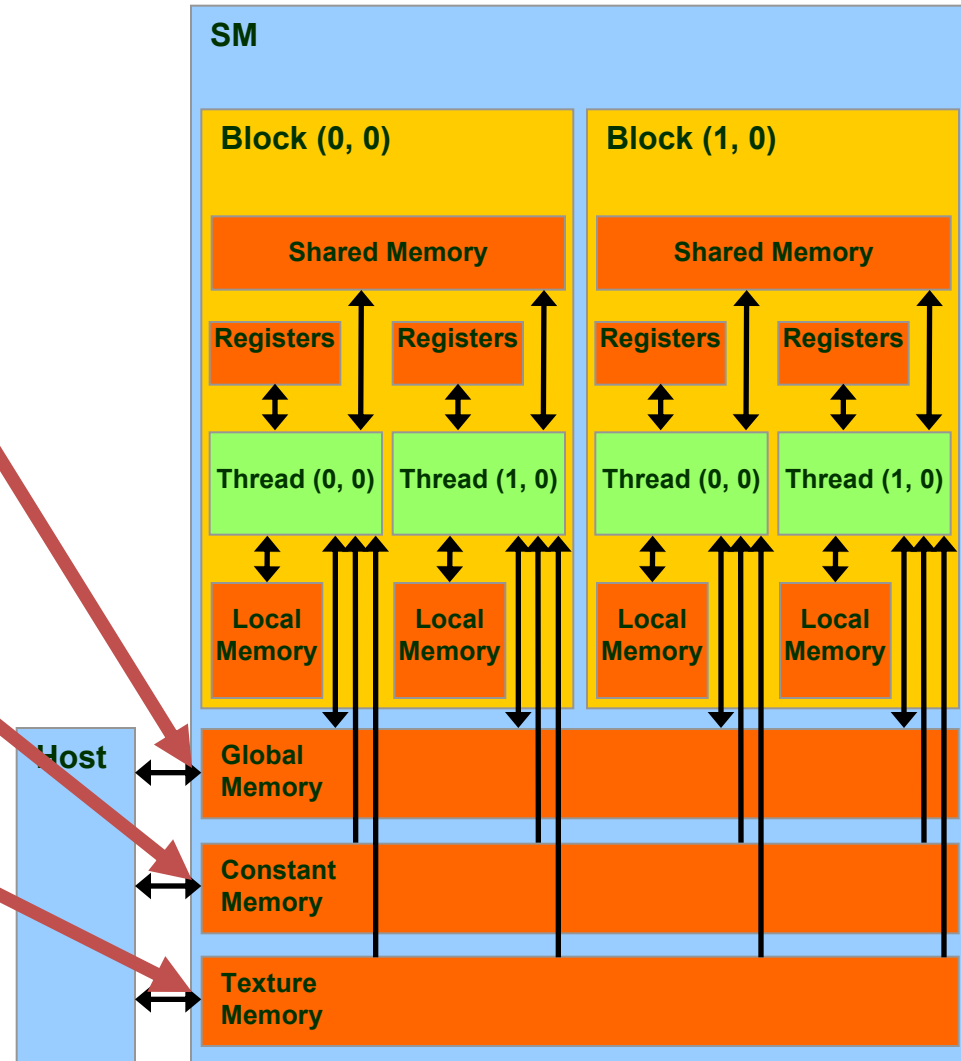
Global, Constant, and Texture Memories

□ Global Memory

- host 和 device之间主要的数
据读写媒介
- 所有线程可见

□ Texture and Constant Memories

- 由host初始化的常量
- 所有线程可见

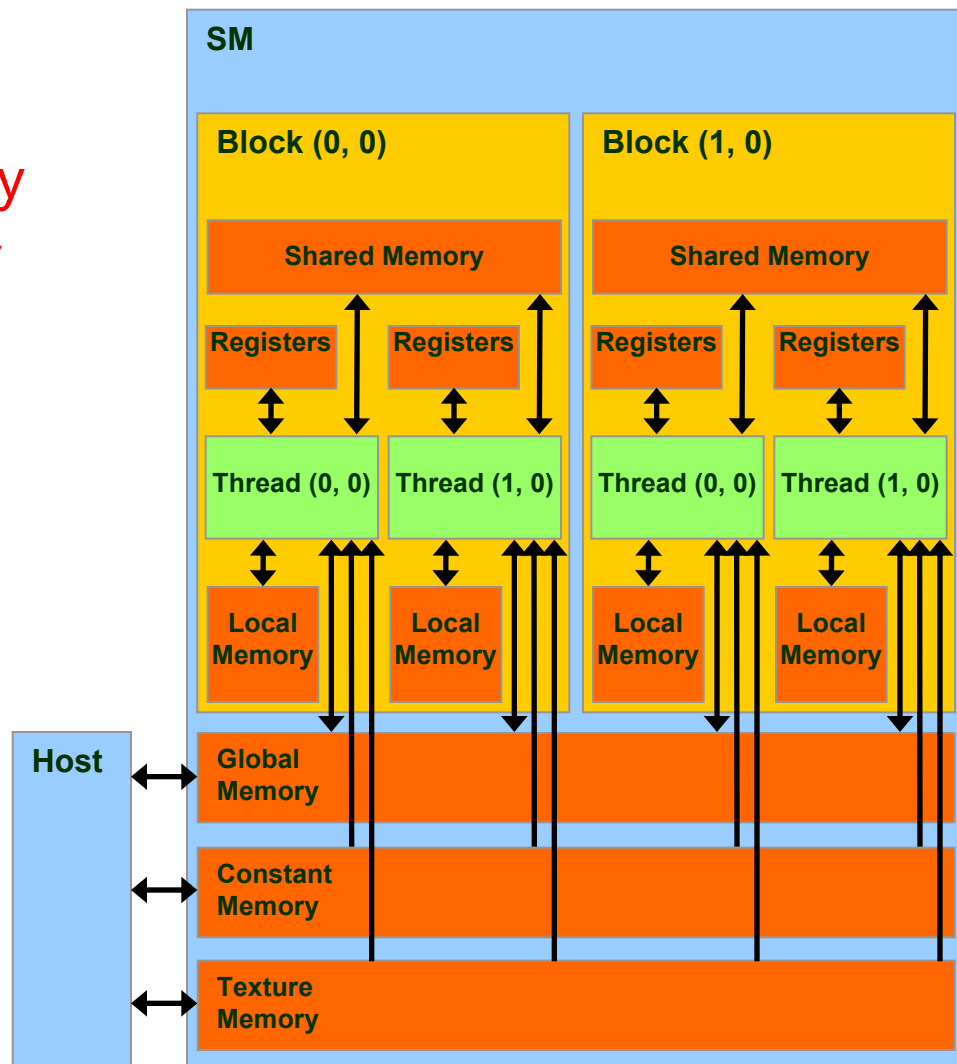


CUDA Device Memory

□ 每个线程可以:

- 读写线程私有的寄存器
- 读写线程私有的local memory
- 读写block的shared memory
- 读写global memory
- 只读全局constant memory
- 只读全局texture memory

□ The host 可以读写 global, constant, 和 texture memories



GPU内存结构

□ 很多线程访问内存，带宽成为瓶颈

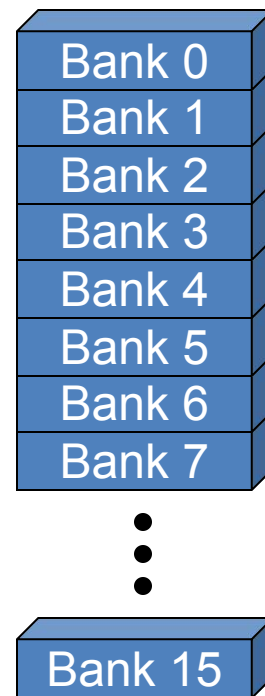
- memory划分成多个 **banks**
- 提高带宽的有效手段

□ 每个bank每个周期只能服务一个请求

- 多个bank之间可以并行访问

□ 多个请求同时访问同一个bank会引发冲突

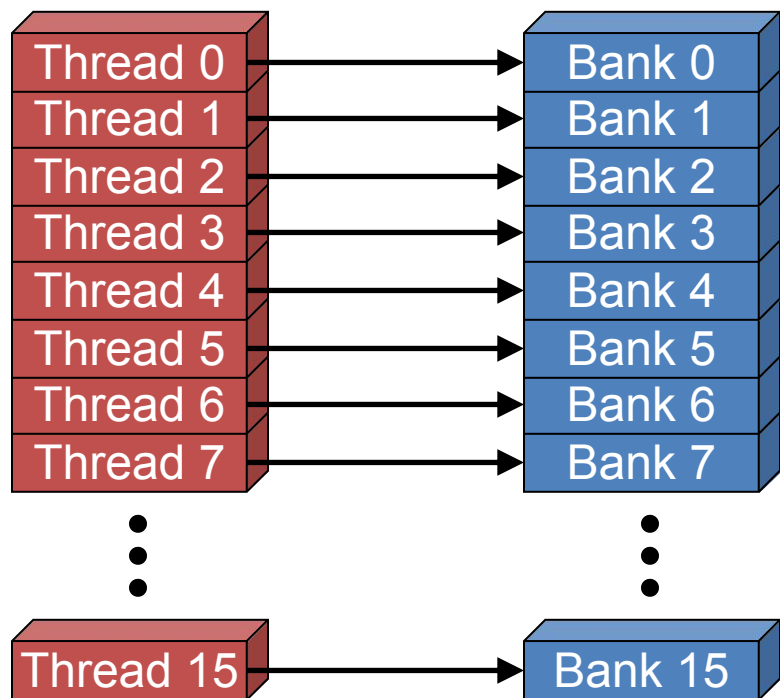
- 冲突访问必须串行



Bank地址映射

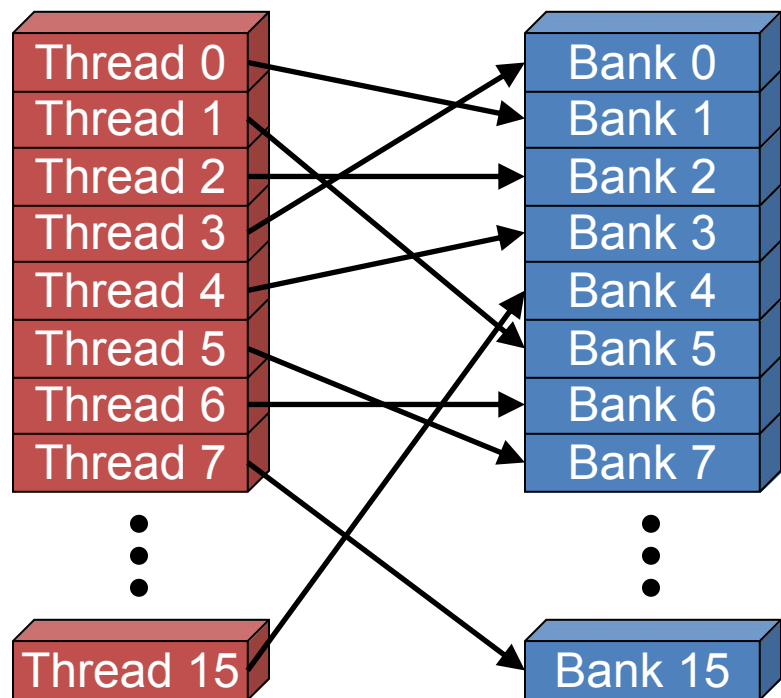
- 无冲突

- Linear addressing
stride == 1

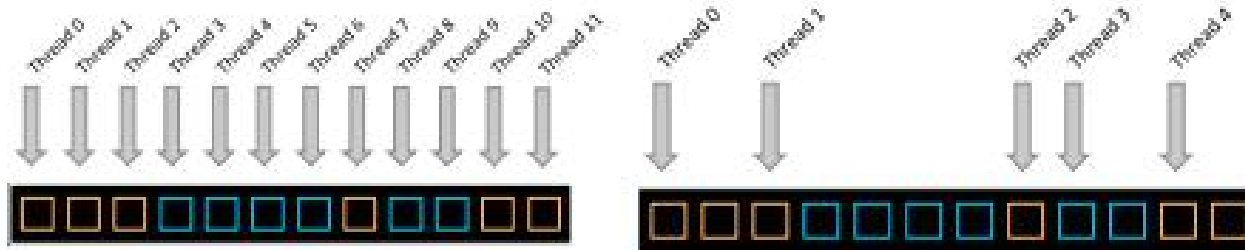


- 无冲突

- Random 1:1 Permutation



Memory 合并



□ 如果线程访问的地址相邻

- 合并逻辑将多个请求合并为一个
- 大大减少访问次数

□ 如果请求不能合并

- 请求将串行化执行
- Warp执行时间将会延长
- 吞吐量下降, 更多能耗

□ GPU架构特点

- 使用大量简单核心
- 采用单指令多线程结构
- 维护大量线程

□ GPU设计挑战

- 分支分歧
- 访存分歧